

Classification Trees

IN5148: Statistics and Data Science with Applications in
Engineering

Alan R. Vazquez
Department of Industrial Engineering

Agenda

1. Introduction
2. Classification Trees
3. Class-Specific Metrics

Introduction

Load the libraries

Before we start, let's import the data science libraries into Python.

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3 import seaborn as sns
4 from sklearn.model_selection import train_test_split, GridSearchCV
5 from sklearn.tree import DecisionTreeClassifier, plot_tree
6 from sklearn.preprocessing import StandardScaler
7 from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
8 from sklearn.metrics import accuracy_score, recall_score, precision_score
```

Here, we use specific functions from the **pandas**, **matplotlib**, **seaborn** and **sklearn** libraries in Python.

Bayes classifier

It turns out that, if we know the true structure of $P(Y = 1 | \mathbf{X} = \mathbf{x})$, we can build a good classification function called the **Bayes classifier**:

$$C(\mathbf{X}) = \begin{cases} 1, & \text{if } P(Y = 1 | \mathbf{X} = \mathbf{x}) > 0.5 \\ 0, & \text{if } P(Y = 1 | \mathbf{X} = \mathbf{x}) \leq 0.5 \end{cases}$$

This function classifies to the most probable class using the conditional distribution $P(Y | \mathbf{X} = \mathbf{x})$.

HOWEVER, we don't (and will never) know the true form of $P(Y = 1 | \mathbf{X} = \mathbf{x})$!

To overcome this issue, we several methods:

- **K-Nearest Neighbours**: Estimate $P(Y = 1 | \mathbf{X} = \mathbf{x})$ directly. Already covered.
- **Classification Trees**: Estimate $P(Y = 1 | \mathbf{X} = \mathbf{x})$ directly. What we will cover today.
- **Ensemble methods**: Estimate $P(Y = 1 | \mathbf{X} = \mathbf{x})$ directly. Later.

Once we estimate $P(Y = 1|\mathbf{X} = \mathbf{x})$ using one of these methods, we plug it into the Bayes classifier:

$$\hat{C}(\mathbf{X}) = \begin{cases} 1, & \text{if } \hat{P}(Y = 1|\mathbf{X} = \mathbf{x}) > 0.5 \\ 0, & \text{if } \hat{P}(Y = 1|\mathbf{X} = \mathbf{x}) \leq 0.5 \end{cases}.$$

where

- $\hat{P}(Y = 1|\mathbf{X} = \mathbf{x})$ is an estimate of $P(Y = 1|\mathbf{X} = \mathbf{x})$.
- $\hat{C}(\mathbf{X})$ is an estimate of the true Bayes Classifier $C(\mathbf{X})$

Example 1: Identifying Counterfeit Banknotes



Dataset

The data is located in the file “banknotes.xlsx”.

```
1 bank_data = pd.read_excel("banknotes.xlsx")
2 # Set response variable as categorical.
3 bank_data['Status'] = pd.Categorical(bank_data['Status'])
4 bank_data.head()
```

	Status	Left	Right	Bottom	Top
0	genuine	131.0	131.1	9.0	9.7
1	genuine	129.7	129.7	8.1	9.5
2	genuine	129.7	129.7	8.7	9.6
3	genuine	129.7	129.6	7.5	10.4
4	genuine	129.6	129.7	10.4	7.7

Create the objects

We use the function `.filter()` from **pandas** to select two predictors: Top and Bottom.

```
1 # Set full matrix of predictors.  
2 X_full = bank_data.filter(['Top', 'Bottom'])  
3  
4 # Set full matrix of responses.  
5 Y_full = bank_data['Status']
```

We also set the target category.

```
1 # Create dummy variables.  
2 Y_dummies = pd.get_dummies(Y_full, dtype = 'int')  
3  
4 # Select target variable.  
5 Y_target_full = Y_dummies['counterfeit']
```

Let's partition the dataset

```
1 X_train, X_valid, Y_train, Y_valid = train_test_split(X_full, Y_target_full,
2                                                         test_size = 0.3,
3                                                         stratify = Y_target_full,
4                                                         random_state=507134)
```

- Thanks to the `stratify` parameter, the function makes a clever partition of the data using the *empirical* distribution of the response.
- Usually, the proportion of the dataset that goes to the validation set is 20% or 30%.

Classification Trees

Decision tree

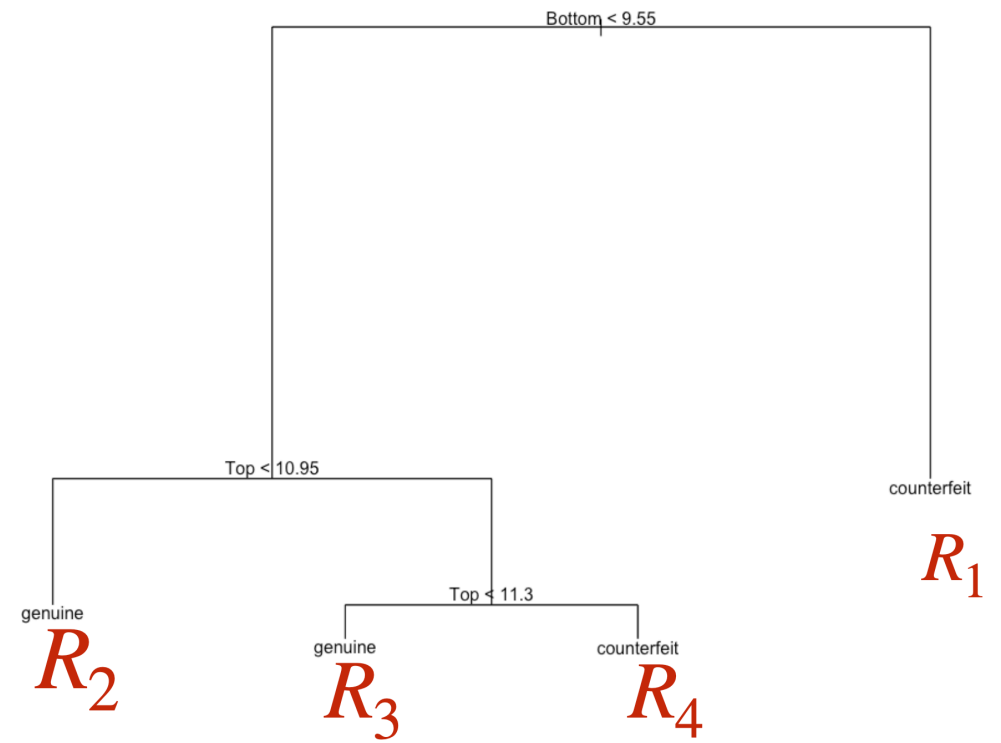
It is a supervised learning algorithm that predicts or classifies observations using a hierarchical tree structure.

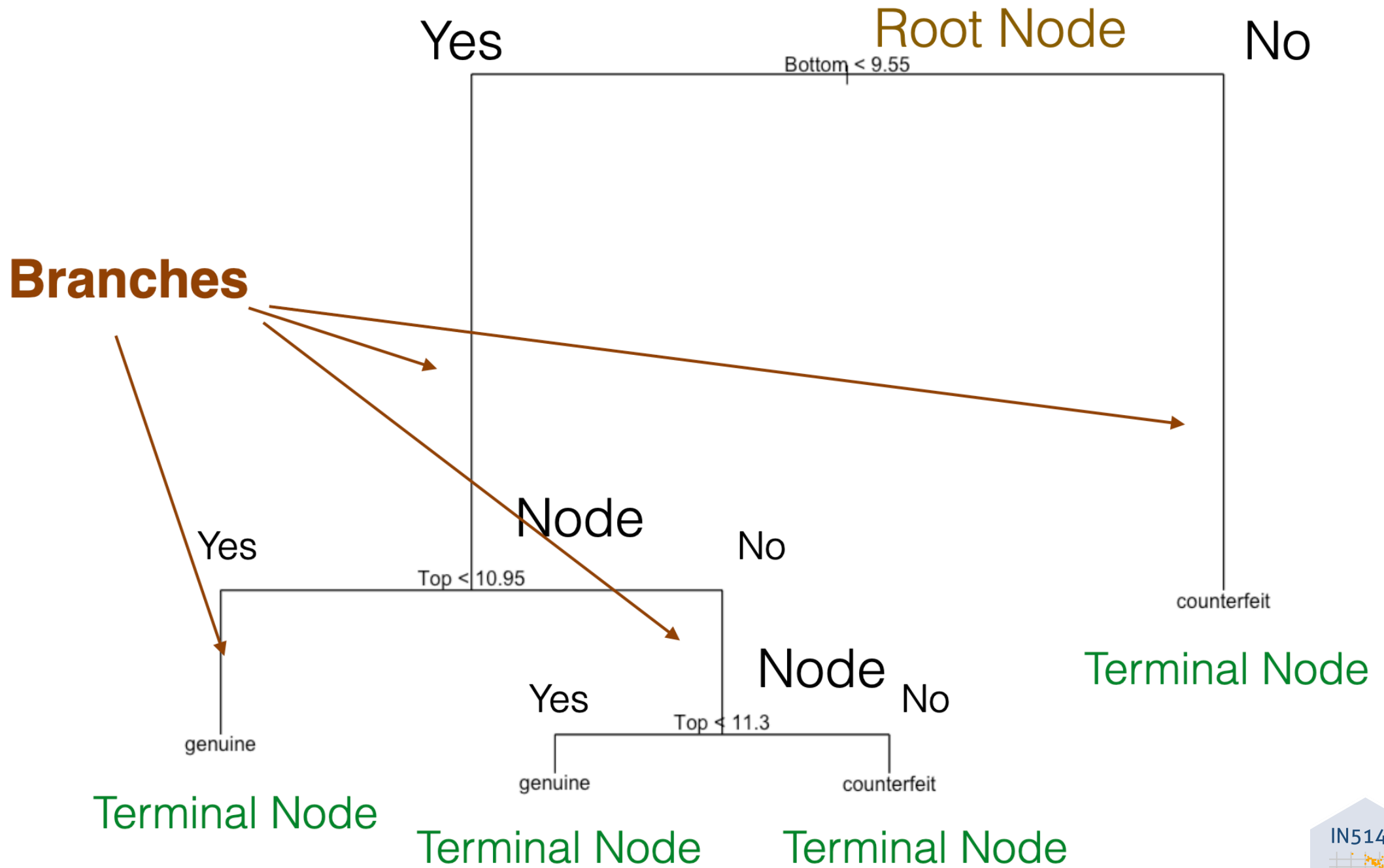
Main characteristics:

- Simple and useful for interpretation.
- Can handle numerical and categorical predictors and responses.
- Computationally efficient.
- Nonparametric technique.

Basic idea of a decision tree

Stratify or segment the predictor space into several simpler regions.





How do you build a decision tree?

Building decision trees involves two main procedures:

1. Grow a large tree.
2. Prune the tree to prevent overfitting.

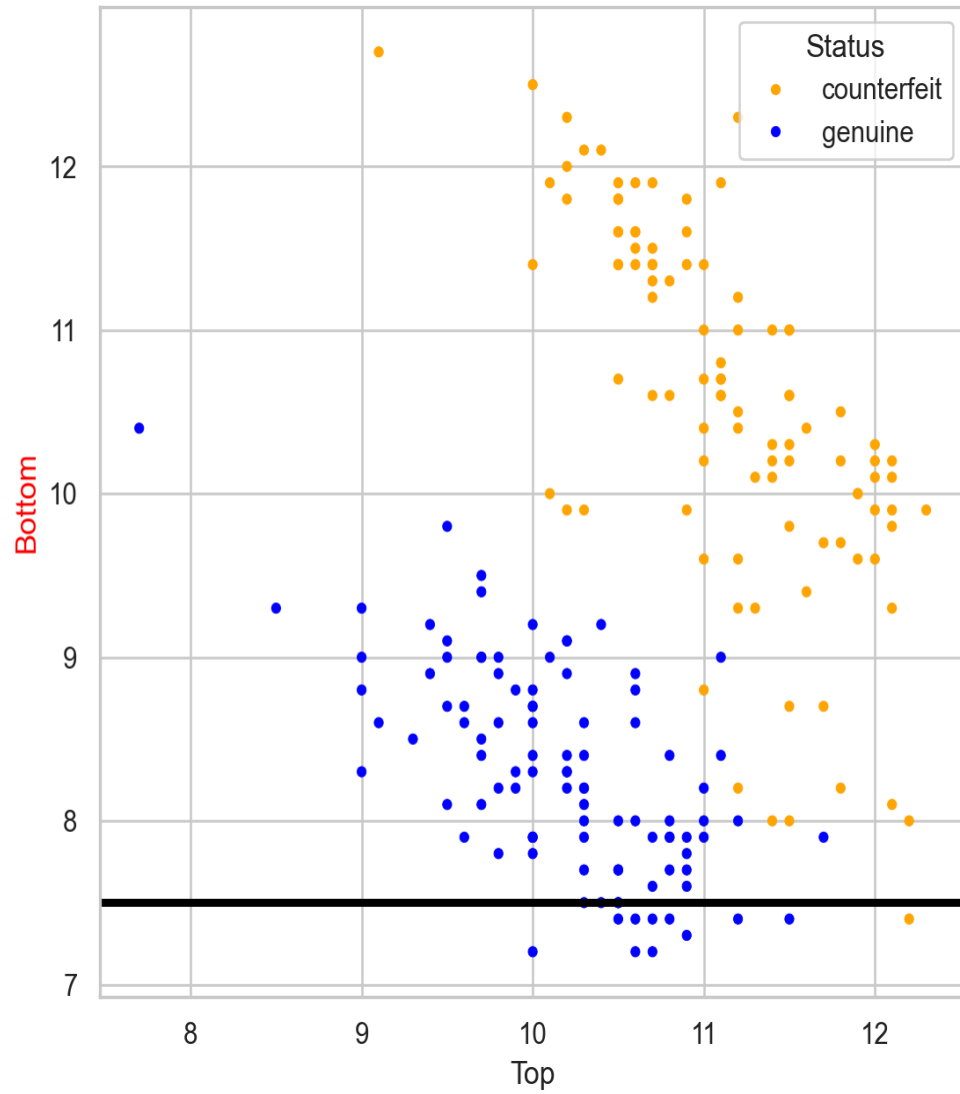
After building a “good” tree, we can predict new observations that are not in the data set we used to build it.

How do we grow a tree?

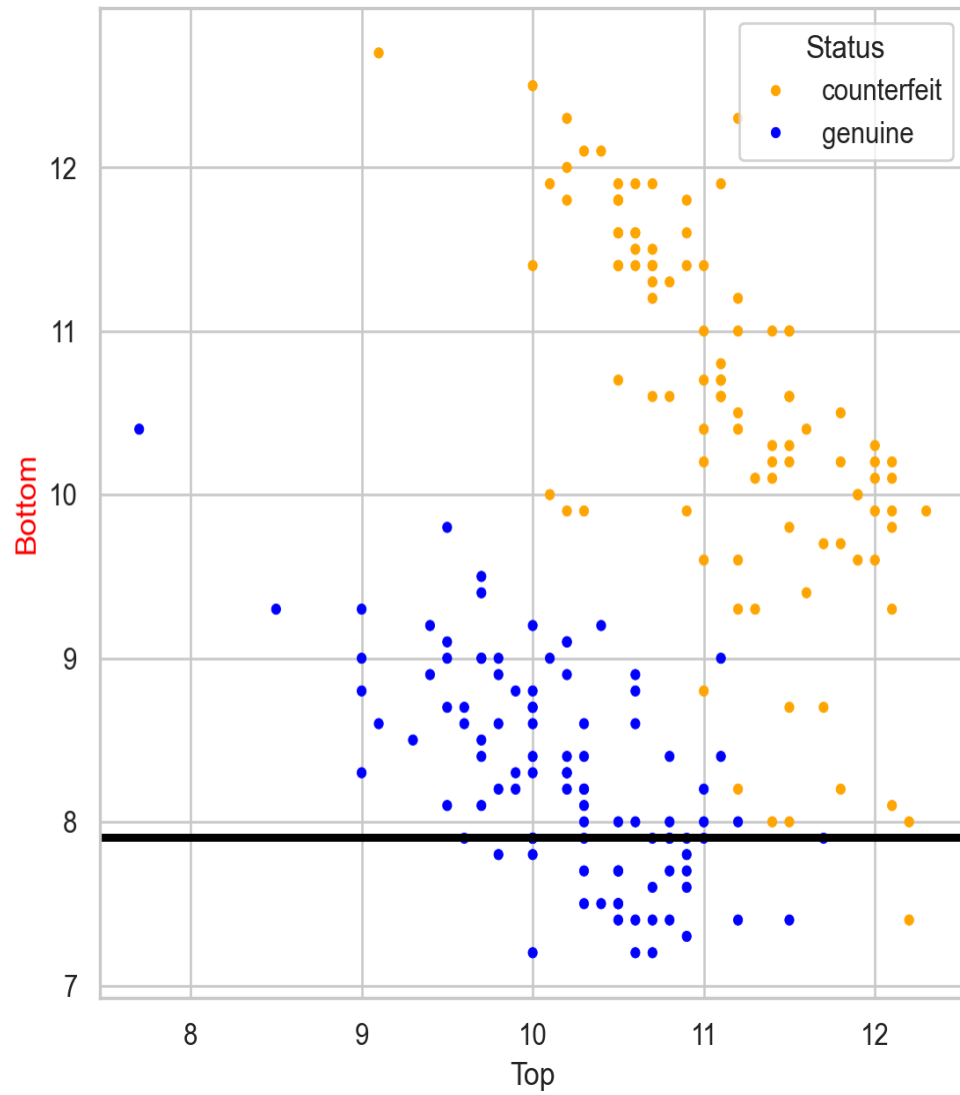
Using the CART algorithm!

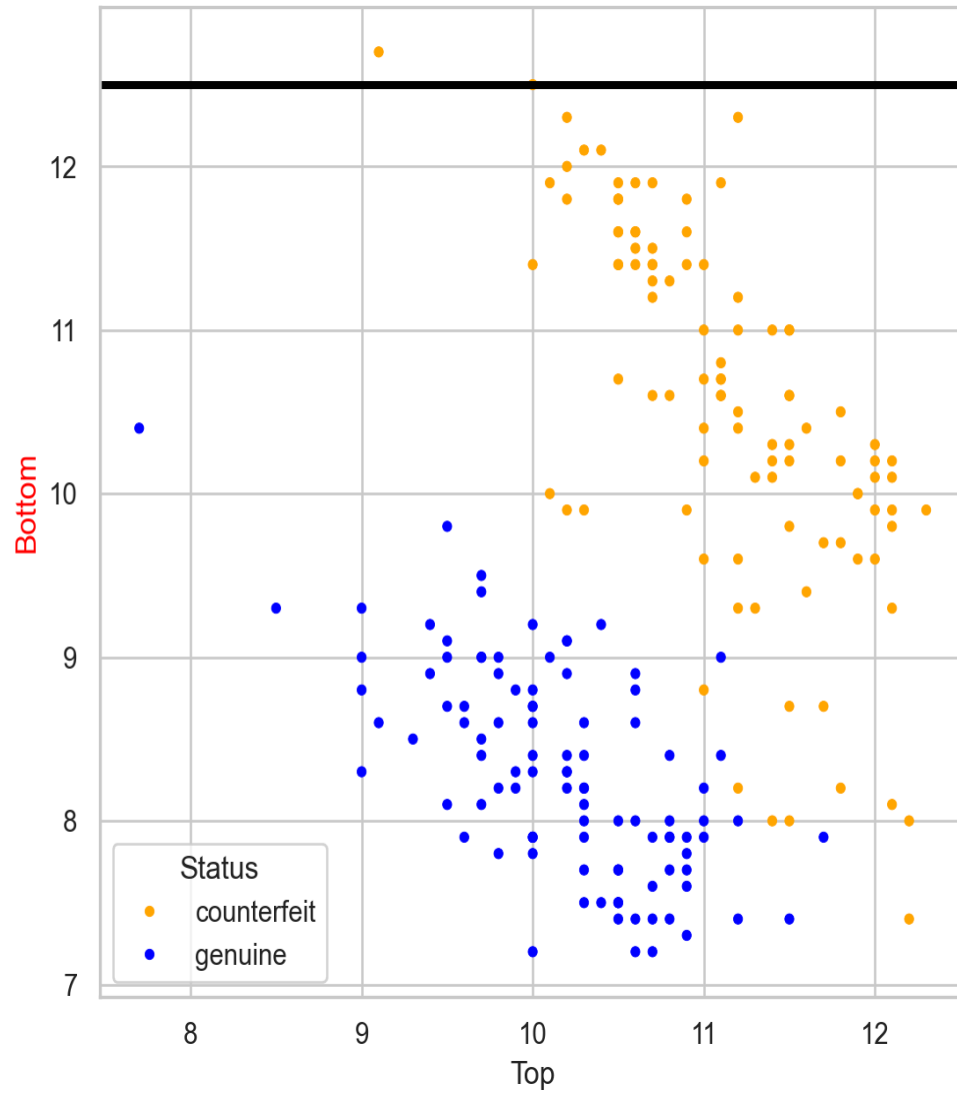
- The algorithm uses a recursive binary splitting strategy that builds the tree using a greedy top-down approach.
- Basically, at a given node, it considers all variables and all possible splits of that variable. Then, for classification, it chooses the best variable and splits it that **minimize** the so-called *impurity*.

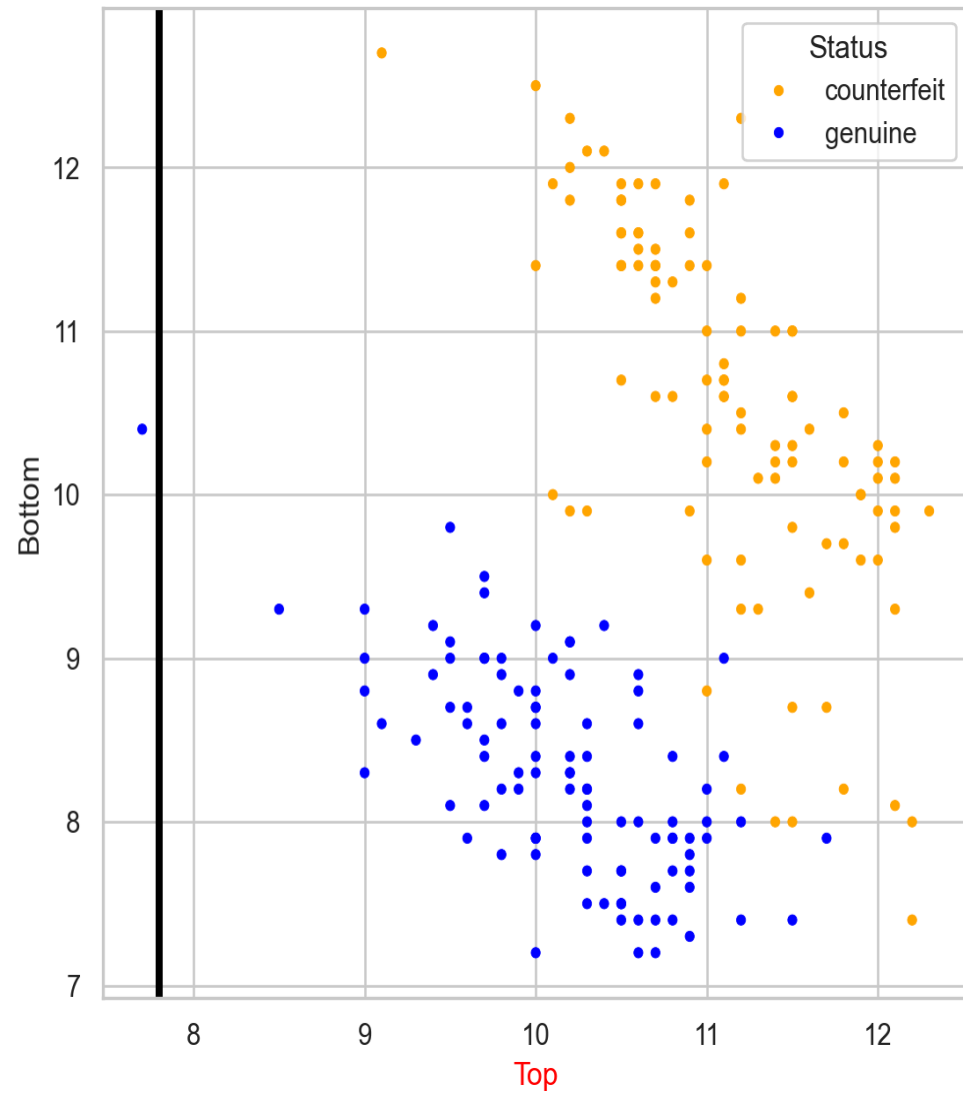


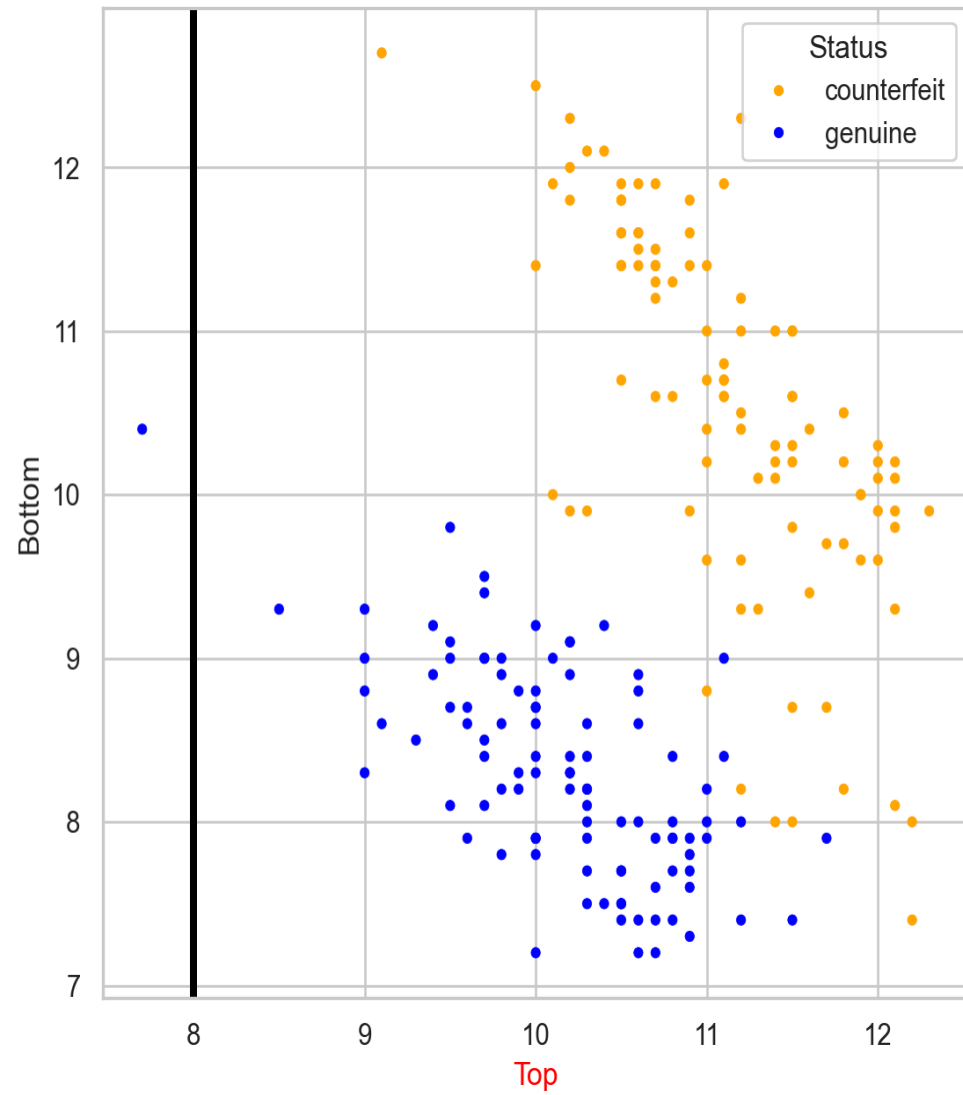


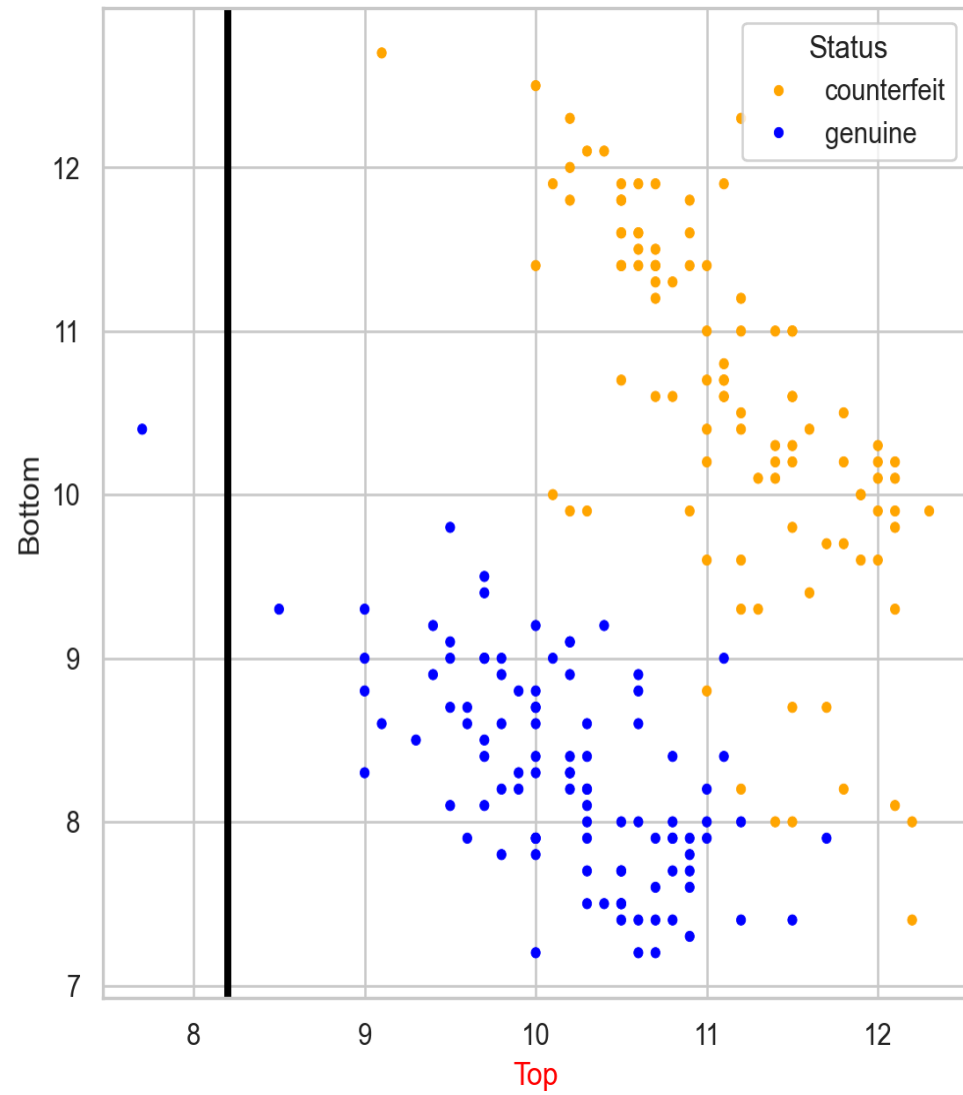




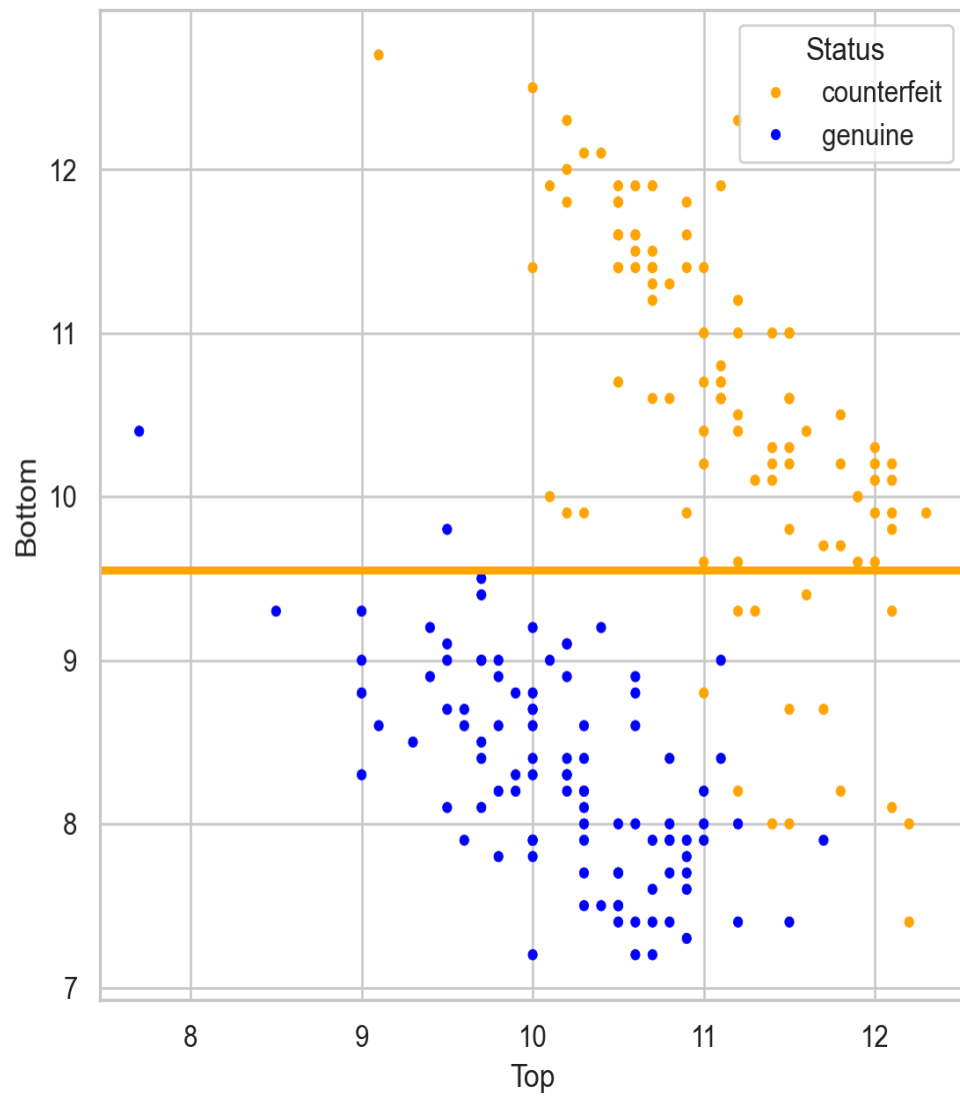




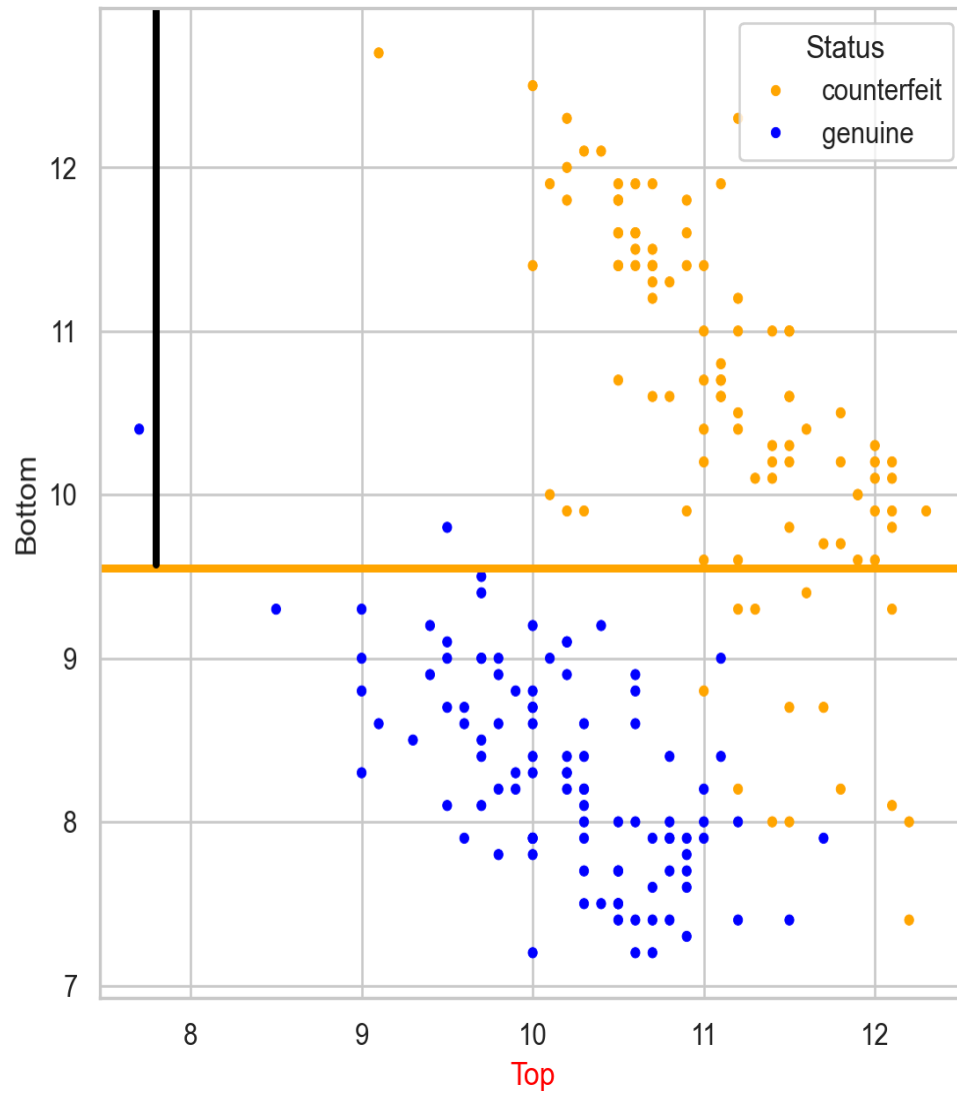






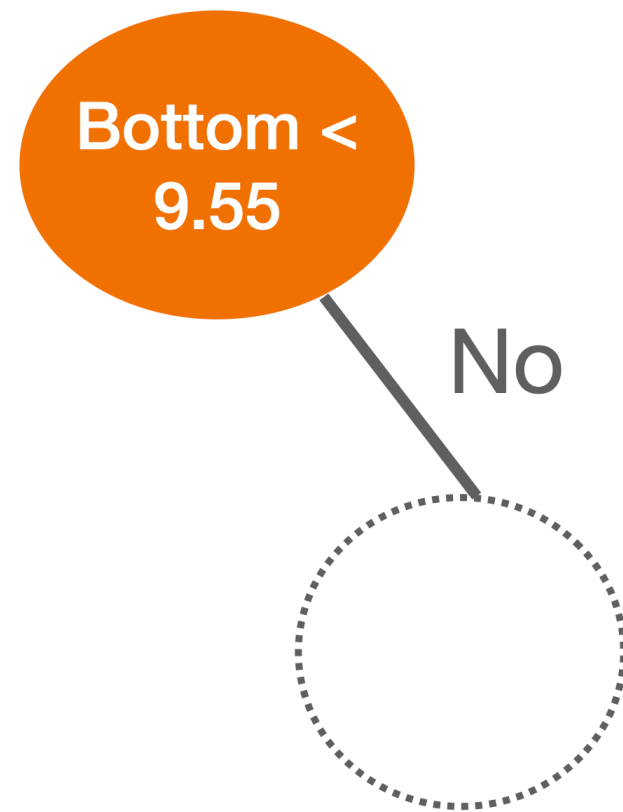
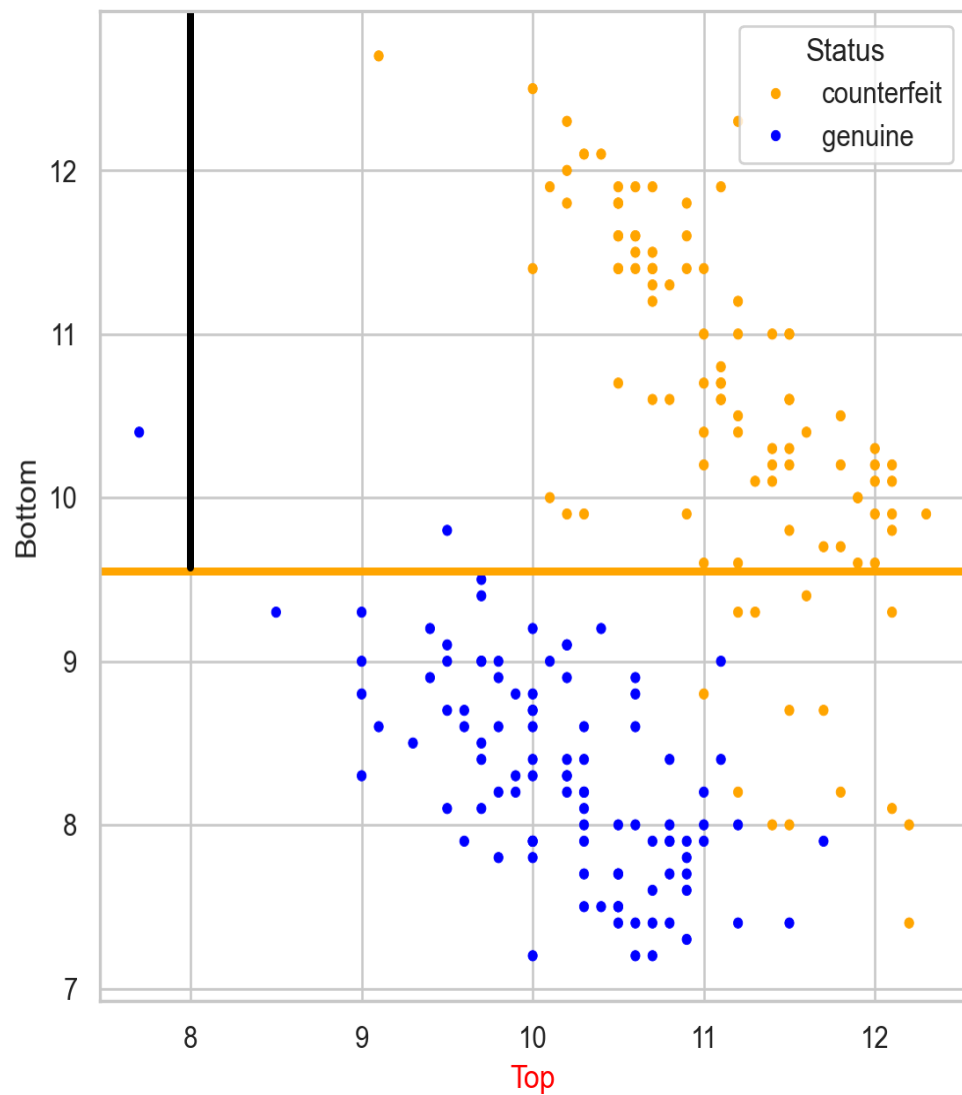


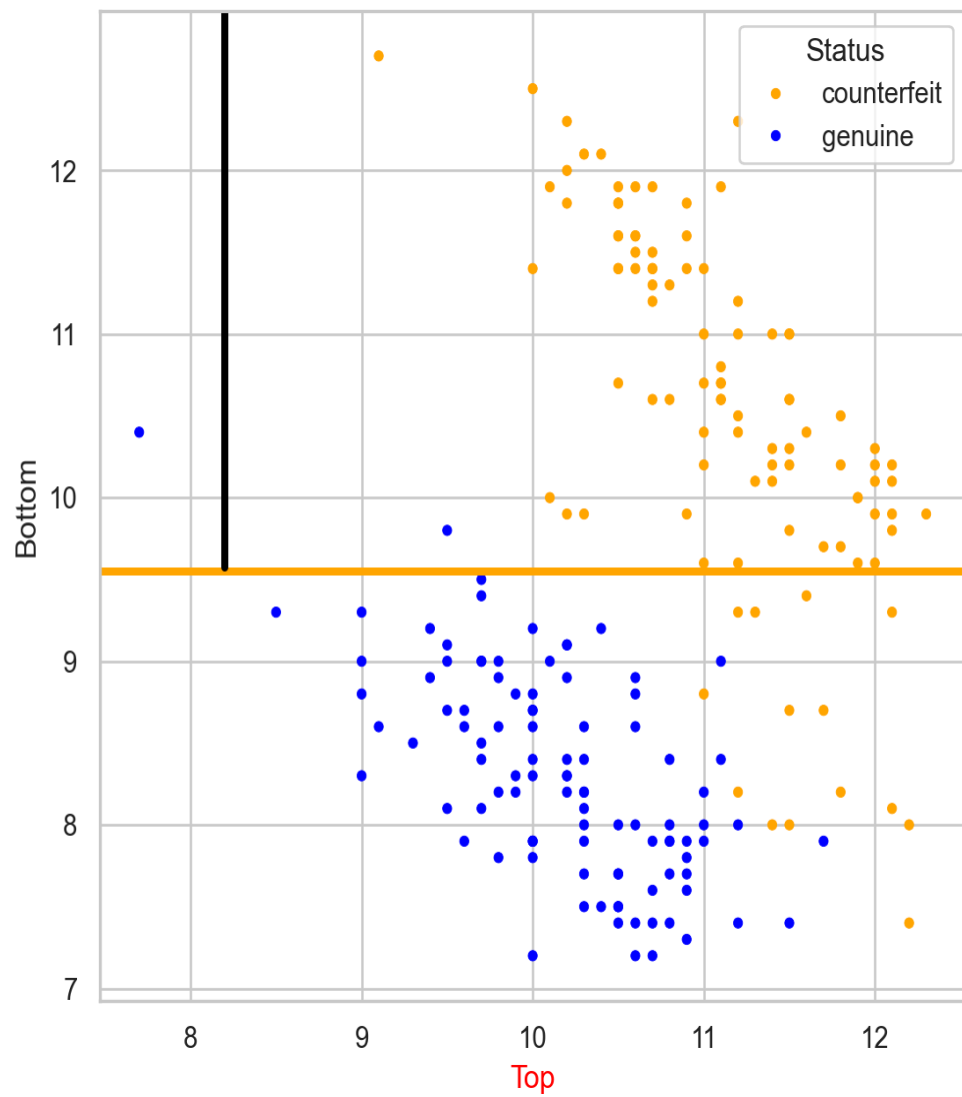
Bottom <
9.55



Bottom <
9.55

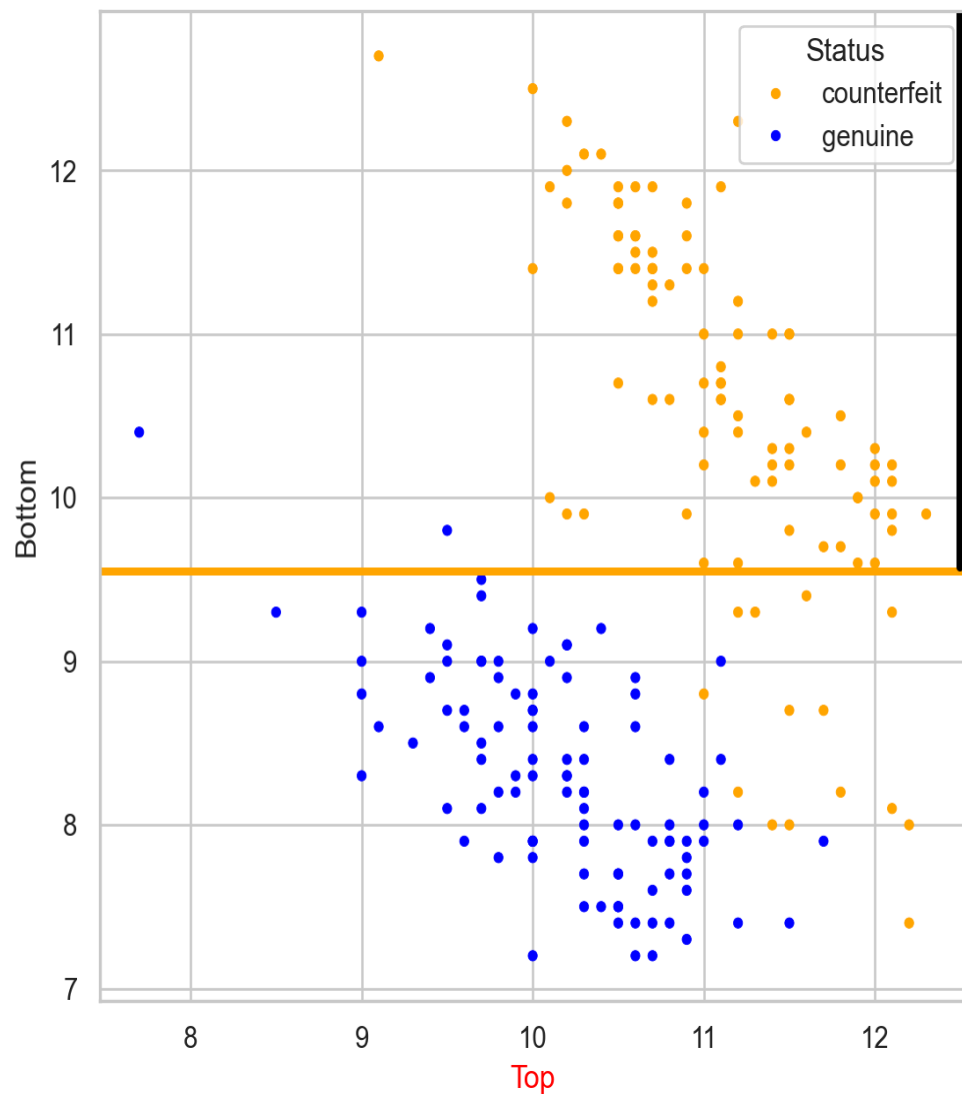
No





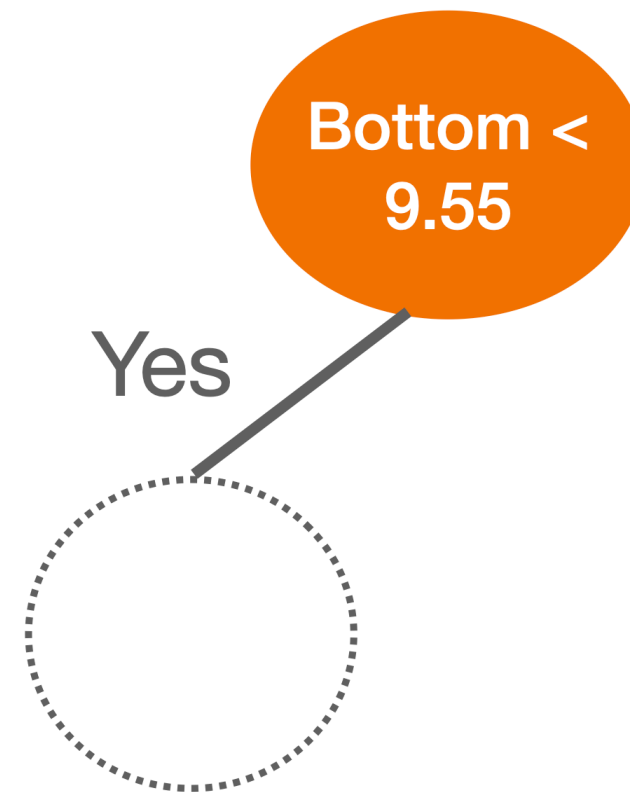
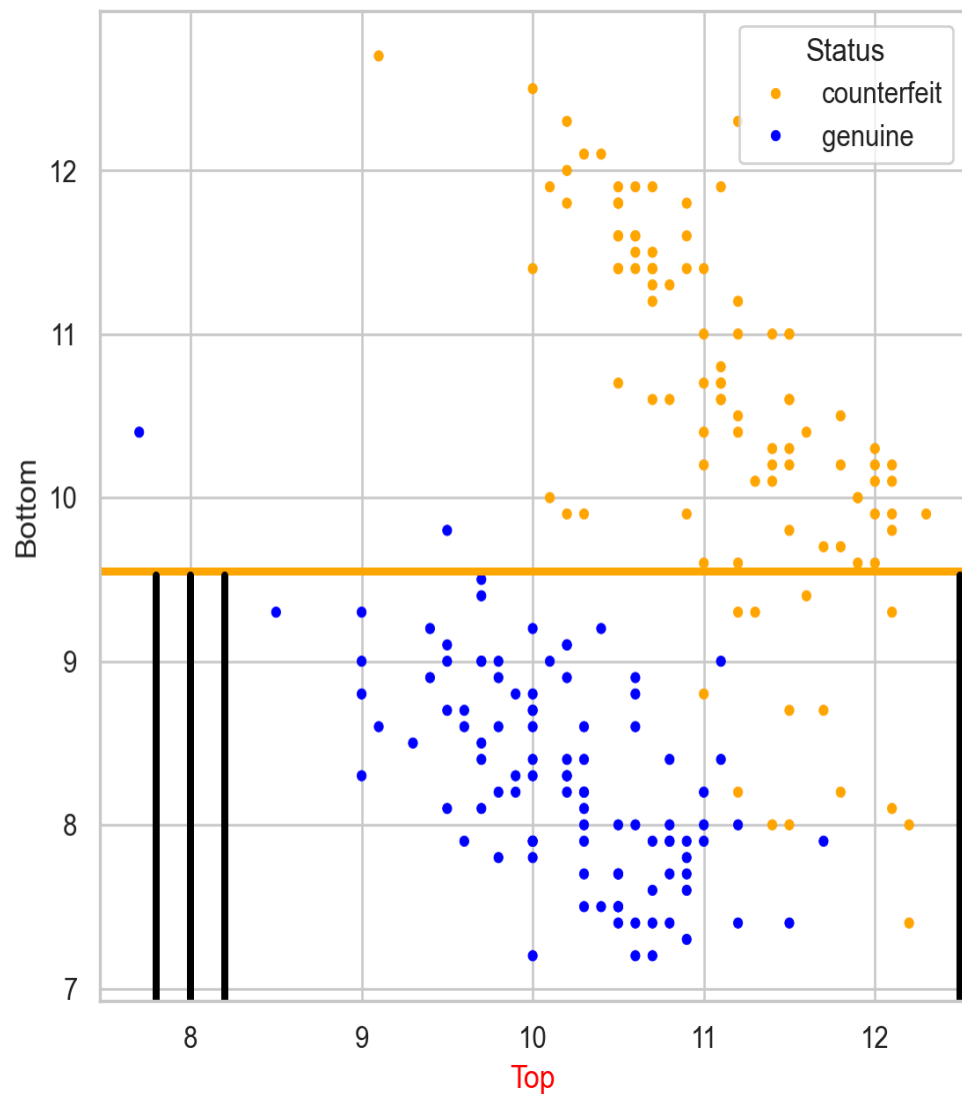
Bottom <
9.55

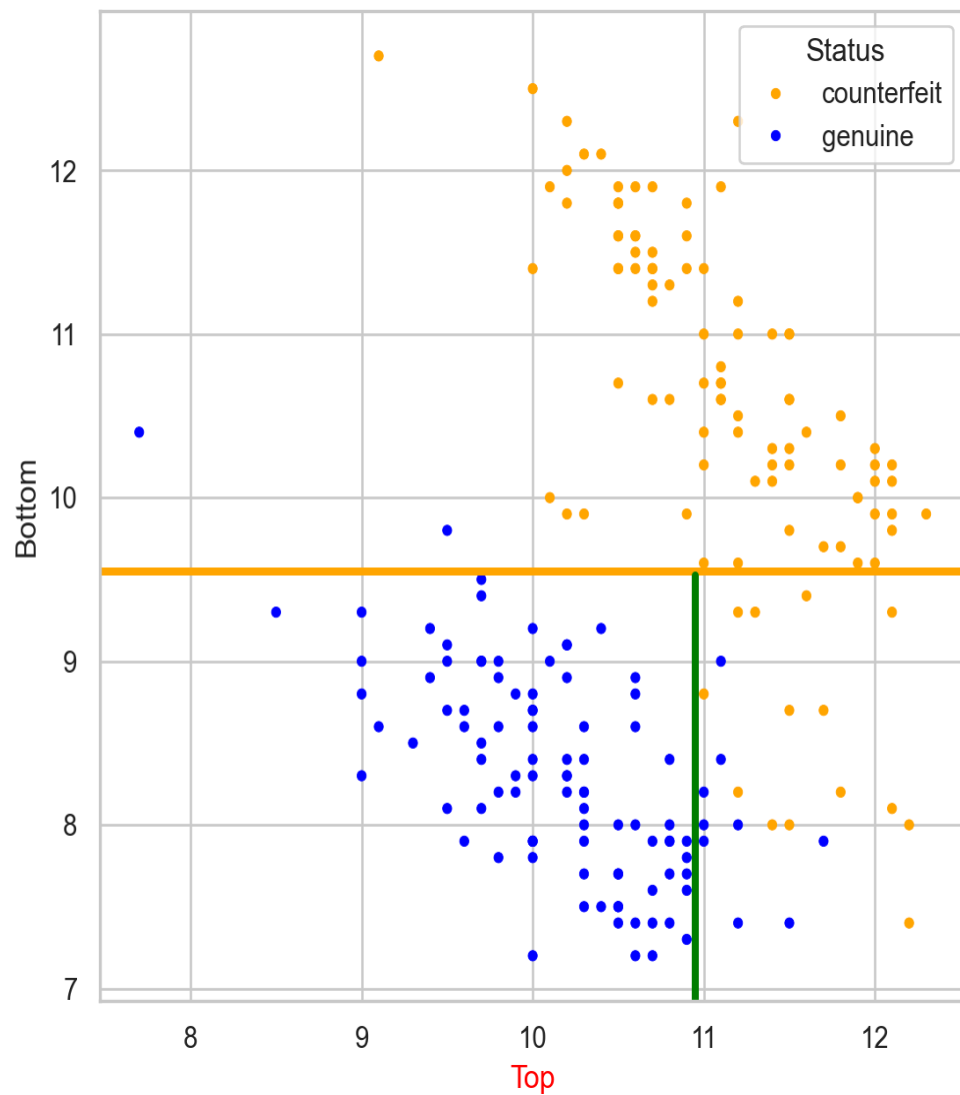
No



Bottom <
9.55

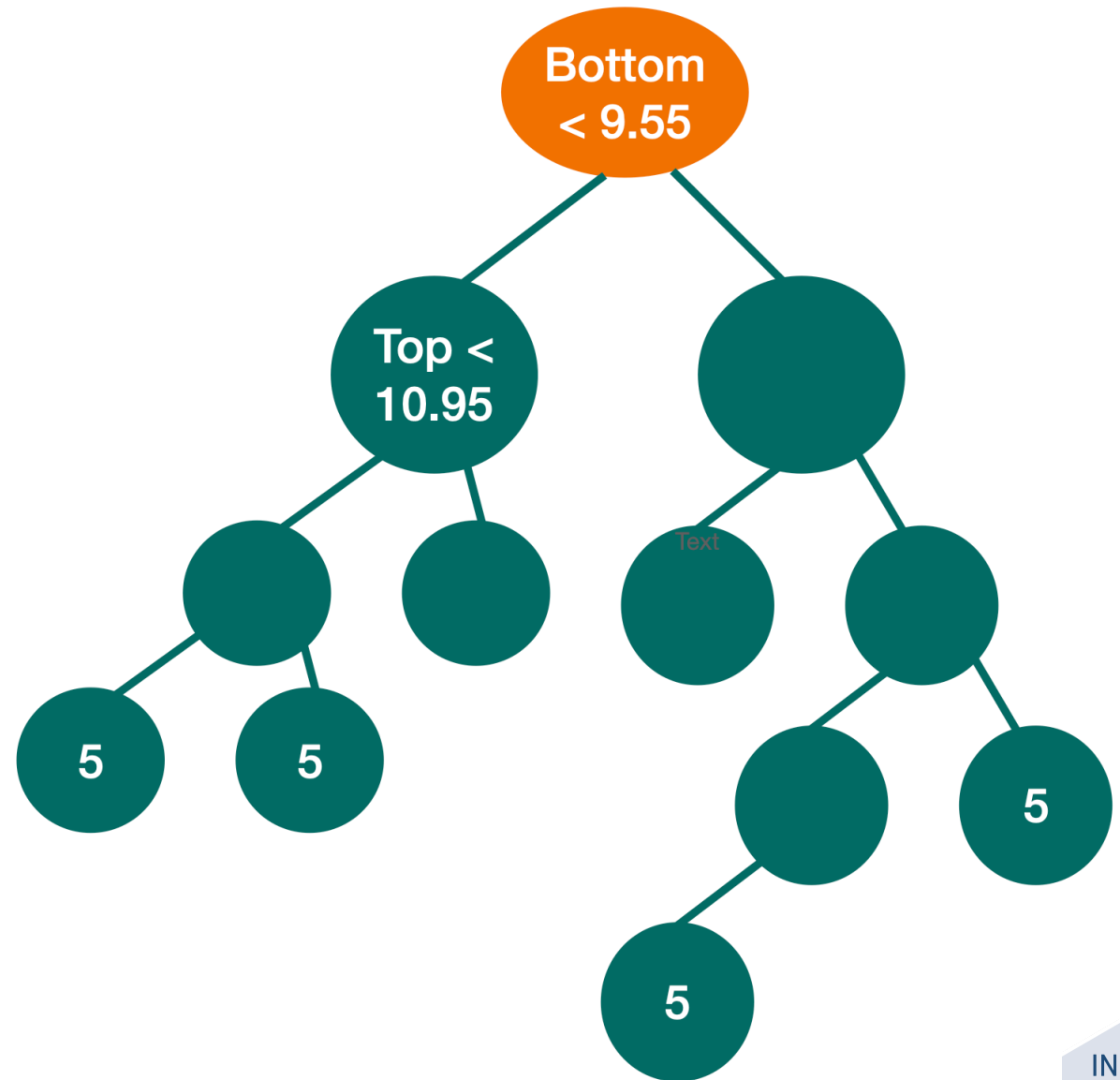
No





We repeat the partitioning process until:

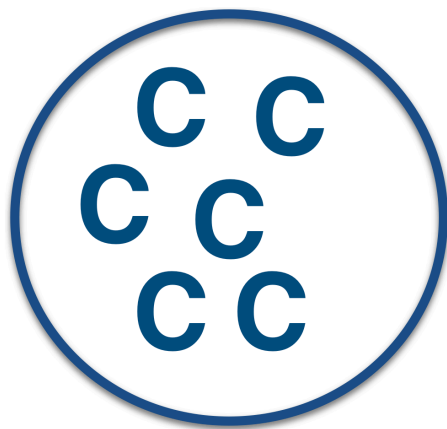
- *impurity* does not improve in any of the terminal nodes, or
- each terminal node has no less than, say, 5 observations.



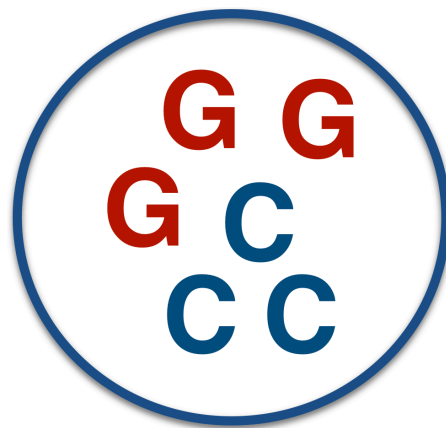
What is impurity?

Node impurity refers to the homogeneity of the response classes at that node.

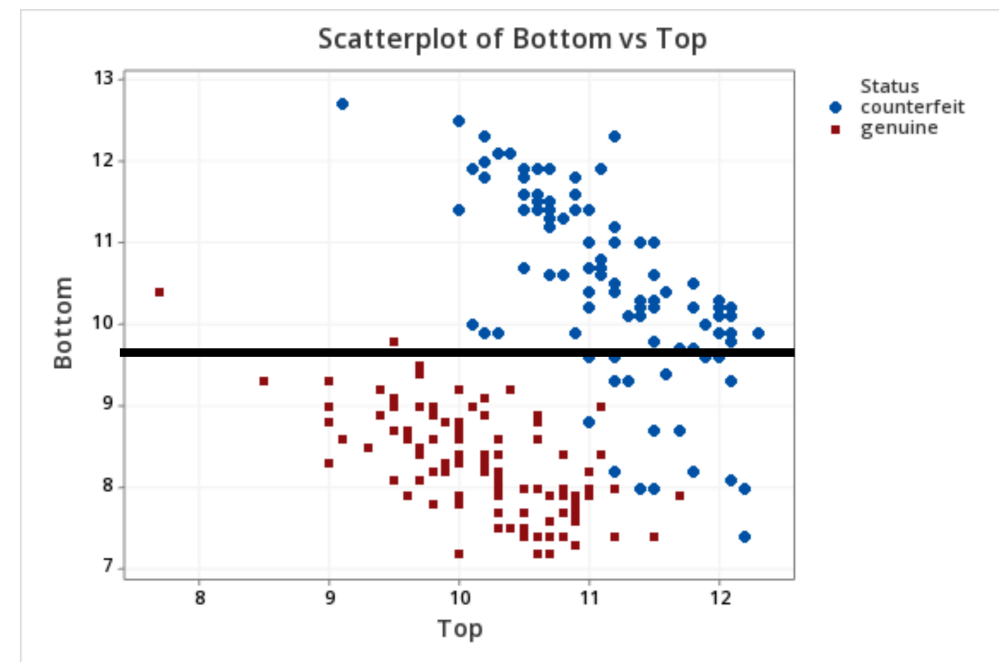
Pure



Impure



C: counterfeit **G: genuine**

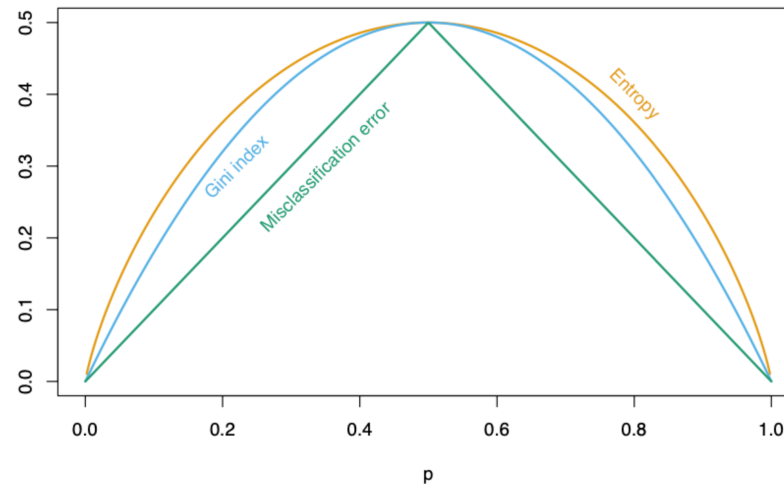
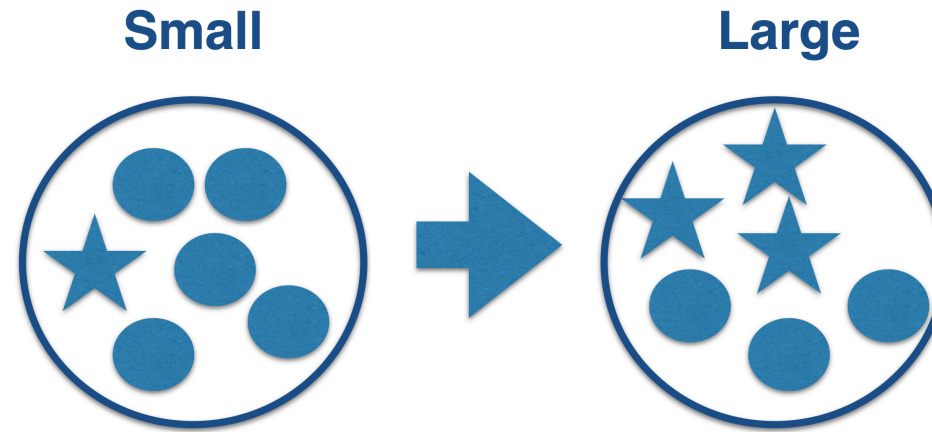


The CART algorithm minimizes impurity between tree nodes.

How do we measure impurity?

There are three different metrics for impurity:

- Misclassification risk.
- Cross entropy.
- Gini impurity index.



p : Proportion of elements in the target class

Mathematically

Let p_1 and p_2 be the proportion of observations in the target and reference class, respectively, in a node.

- Misclassification risk: $1 - \max\{p_1, p_2\}$
- Cross entropy: $-(p_1 \log(p_1) + p_2 \log(p_2))$
- Gini impurity index: $p_1(1 - p_1) + p_2(1 - p_2)$

In Python

In Python, we use the `DecisionTreeClassifier()` and `fit()` functions from **scikit-learn** to train a classification tree.

```
1 # We tell Python we want a classification tree
2 clf = DecisionTreeClassifier(min_samples_leaf = 5, ccp_alpha=0,
3                             random_state=507134)
4
5 # We train the classification tree using the training data.
6 clf.fit(X_train, Y_train)
```

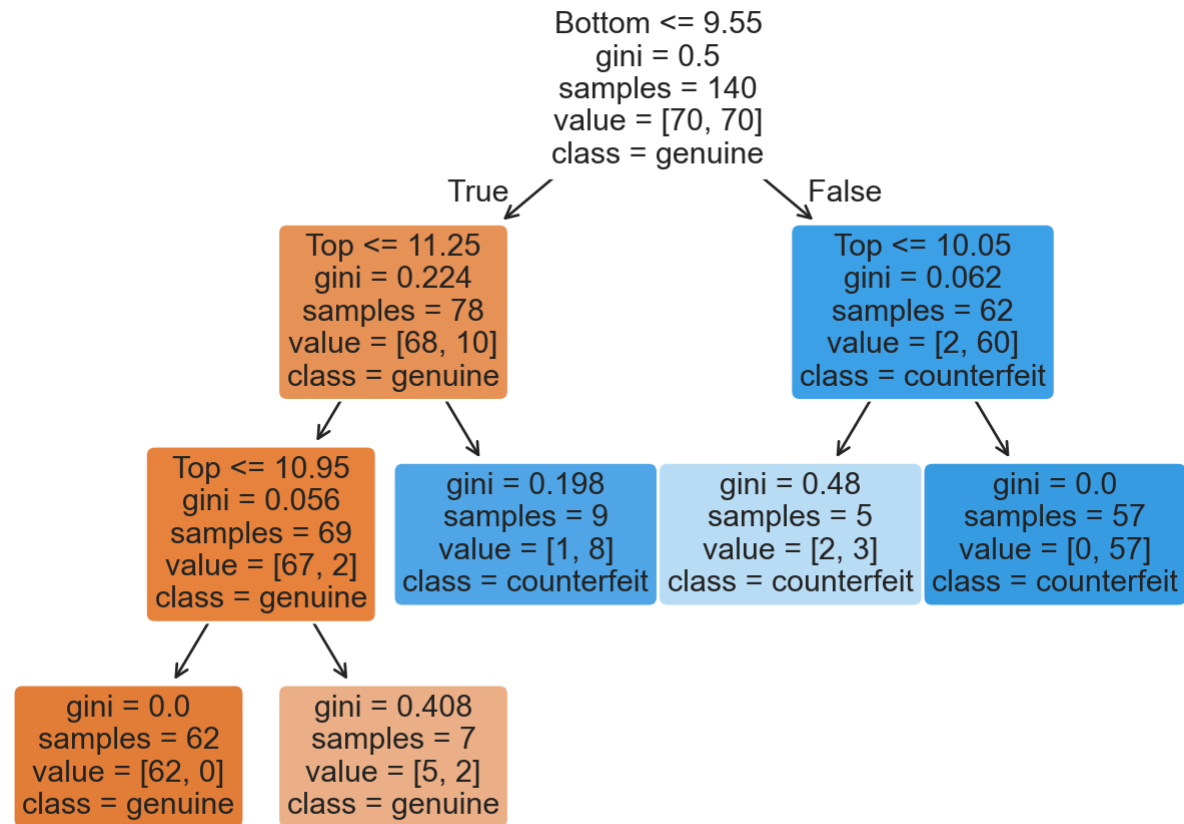
The parameter `min_samples_leaf` controls the minimum number of observations in a terminal node, and the `cc_alpha` controls the tree complexity (to be described later). The parameter `random_state` allows you to reproduce the same tree in different runs of the Python code.

Plotting the tree

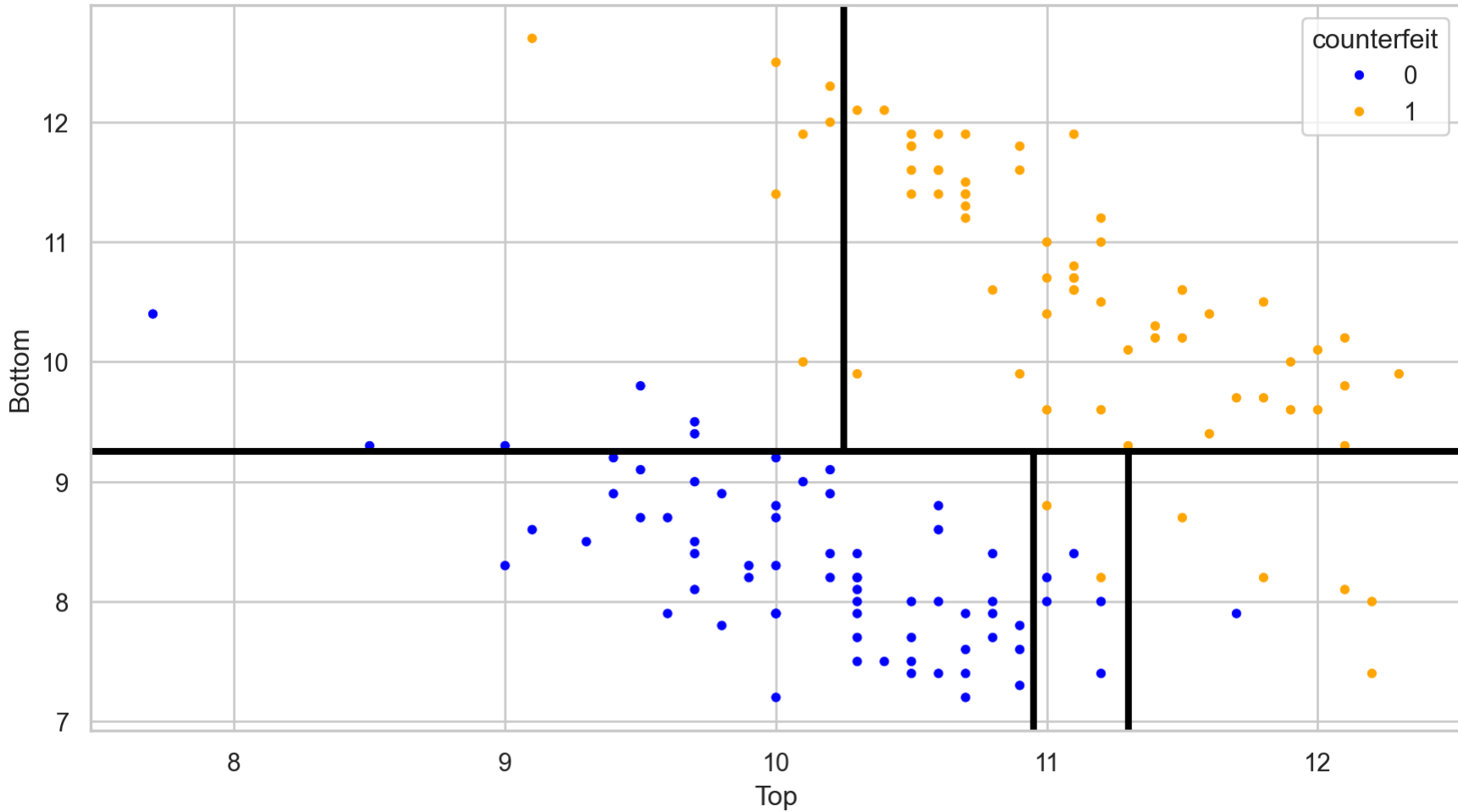
To see the decision tree, we use the `plot_tree` function from **scikit-learn** and some commands from **matplotlib**.

Specifically, we use the `plt.figure()` functions to define the size of the figure and `plt.show()` to display the figure.

```
1 plt.figure(figsize=(6, 6))
2 plot_tree(clf, feature_names = X_train.columns,
3           class_names=["genuine", "counterfeit"], filled=True, rounded=True)
4 plt.show()
```



The tree in the predictor space



Estimated conditional probabilities

After training a classification tree, we can calculate the estimated conditional probability $\hat{P}(Y = 1 | \mathbf{X} = \mathbf{x})$.

To this end, let

- $\hat{p}_1(\mathbf{x}) = \hat{P}(Y = 1 | \mathbf{X} = \mathbf{x})$
- $\hat{p}_0(\mathbf{x}) = 1 - \hat{p}_1(\mathbf{x})$

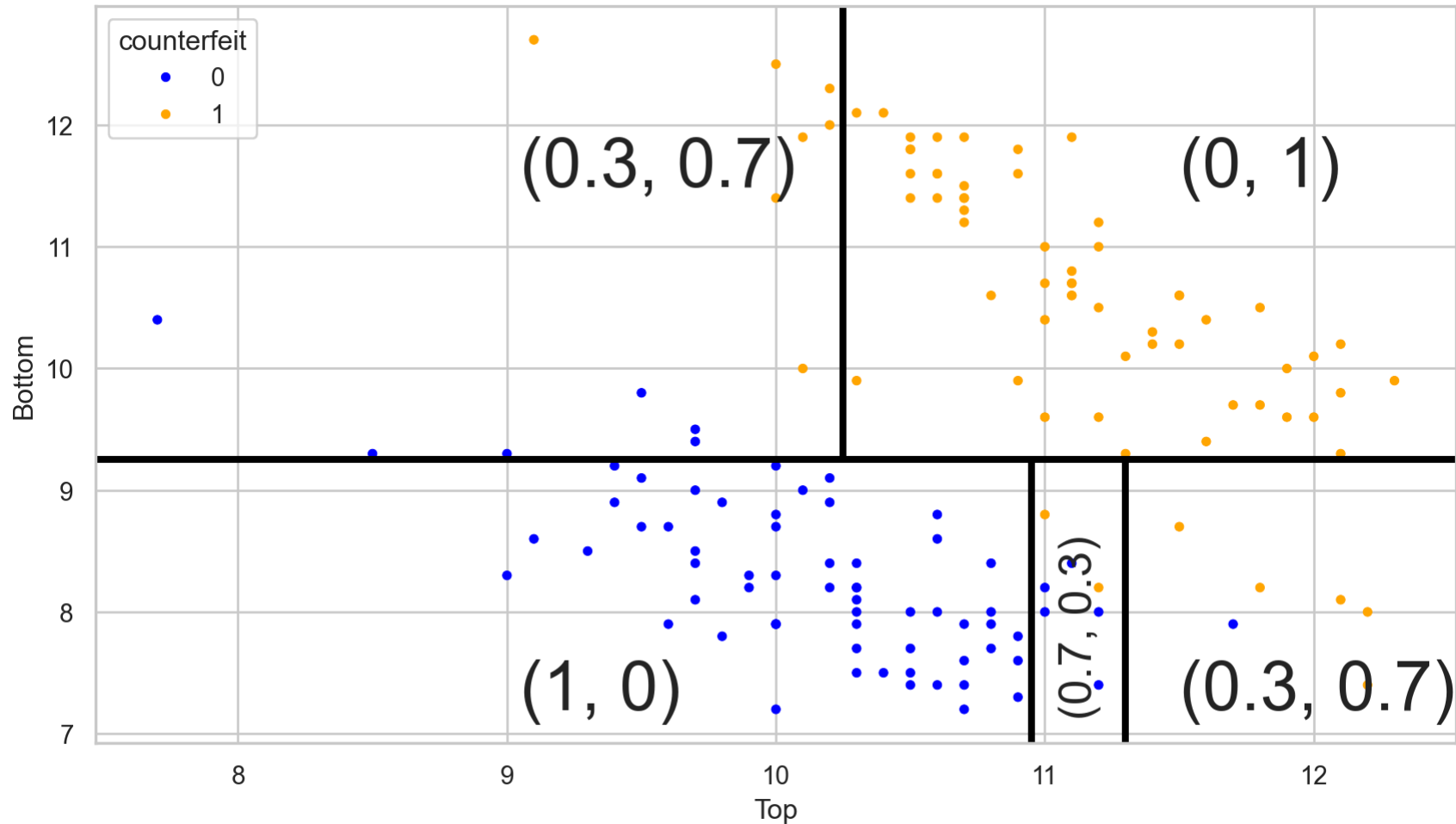
be the estimated probabilities that $\mathbf{x}$ belongs to class 1 or 0.

Essentially, we calculate $\hat{p}_1(\mathbf{x})$ and $\hat{p}_0(\mathbf{x})$ as follows:

1. Select the region or terminal node where $\mathbf{x}$ belongs.
2. $\hat{p}_1(\mathbf{x})$ is the proportion of observations in *that* terminal node that belong to class 1. $\hat{p}_0(\mathbf{x})$ is the proportion of observations that belong to class 0.

Visually

Let $(\hat{p}_0(\mathbf{x}), \hat{p}_1(\mathbf{x}))$ be the estimated probabilities.



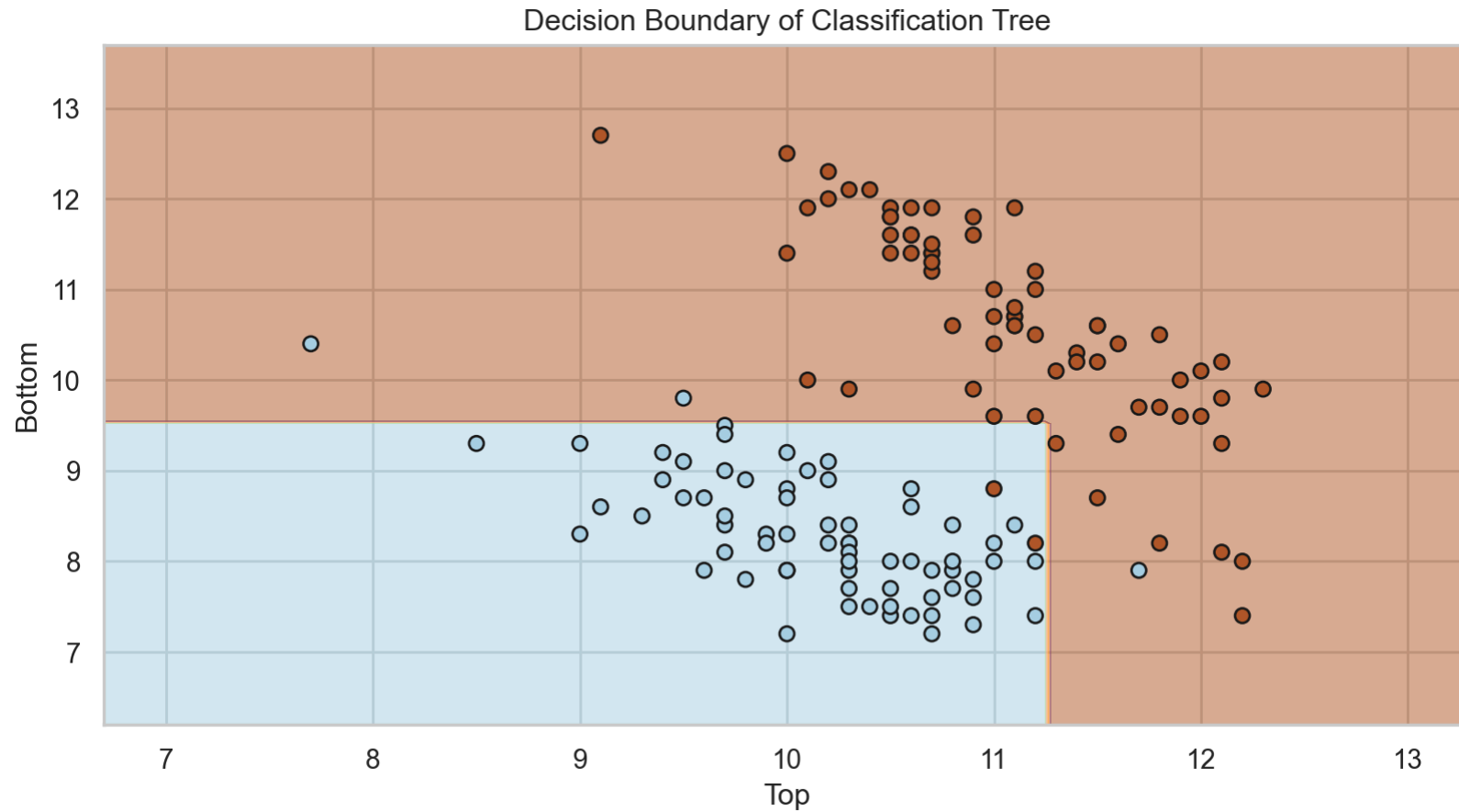
Estimated Bayes Classifier

Once we have estimated the conditional probability using the classification tree, we plug it into the Bayes classifier to have our approximated function:

$$\hat{C}(\mathbf{x}) = \begin{cases} 1, & \text{if } \hat{p}_1(\mathbf{x}) > 0.5 \\ 0, & \text{if } \hat{p}_1(\mathbf{x}) \leq 0.5 \end{cases},$$

where $\hat{p}_1(\mathbf{x})$ depends on the region or terminal node $\mathbf{x}$ falls in.

Simplified decision boundary



Implementation details

- Categorical predictors with unordered levels $\{A, B, C\}$. We order the levels in a specific way (works for binary and regression problems).
- Predictors with missing values. For quantitative predictors, we use multiple imputation. For categorical predictors, we create a new “NA” level.
- Tertiary or quartary splits. There is not much improvement.
- Diagonal splits (using a linear combination for partitioning). These can lead to improvement, but they impair interpretability.

Classification Metrics

Evaluation

We evaluate a classification tree by classifying observations that were not used for training.

That is, we use the classifier to predict categories in the validation data set using only the predictor values from this set.

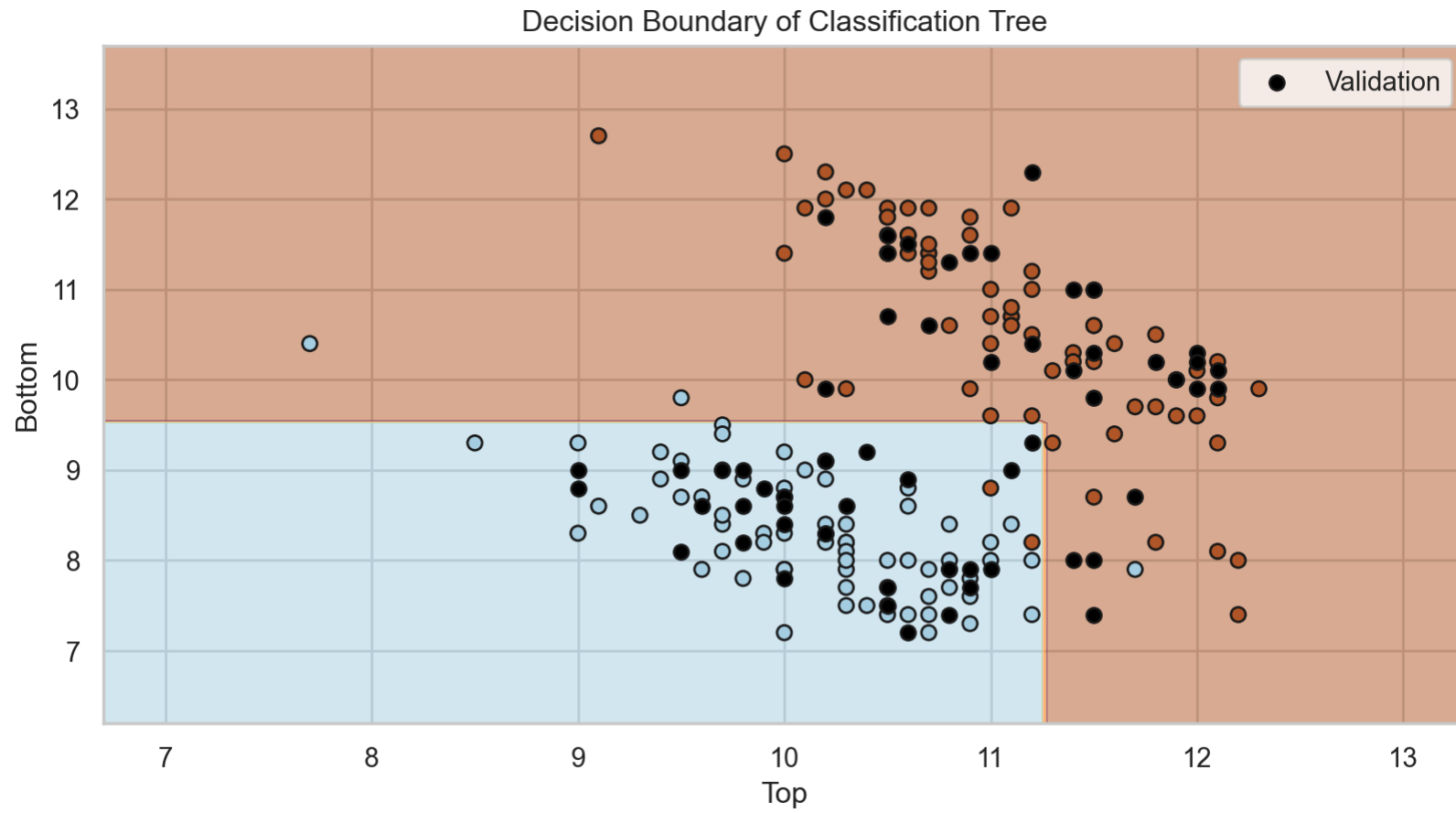
In Python, we use the commands:

```
1 # Predict classes.  
2 predicted_class = clf.predict(X_valid)
```

The `predict()` function generates actual classes to which each observation was assigned.

```
1 predicted_class
```

```
array([0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 1, 0, 1, 0, 0, 1, 1, 1, 0, 0,  
       0, 1, 0, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 1, 0, 1, 0, 0, 0, 1, 1, 0,  
       0, 0, 1, 0, 1, 0, 0, 0, 1, 1, 1, 1, 0, 1, 1, 1])
```



The `predict_proba()` function generates the probabilities used by the algorithm to assign the classes.

```
1 # Predict probabilities.  
2 predicted_probability = clf.predict_proba(X_valid)  
3 predicted_probability
```

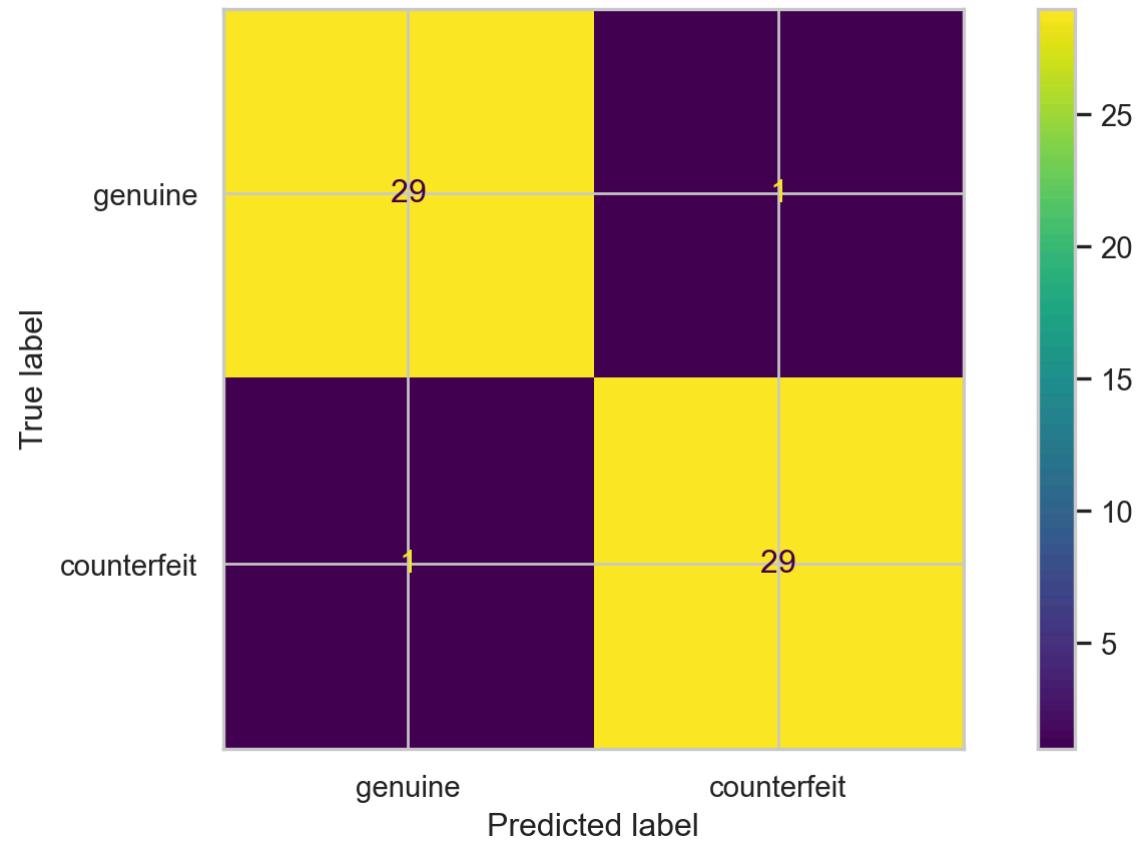
```
array([[1.         , 0.         ],  
       [1.         , 0.         ],  
       [1.         , 0.         ],  
       [1.         , 0.         ],  
       [0.         , 1.         ],  
       [0.         , 1.         ],  
       [0.         , 1.         ],  
       [0.         , 1.         ],  
       [1.         , 0.         ],  
       [1.         , 0.         ],  
       [1.         , 0.         ],  
       [1.         , 0.         ],  
       [0.         , 1.         ],  
       [1.         , 0.         ],  
       ...])
```

Confusion matrix

- Table to evaluate the performance of a classifier.
- Compares actual values with the predicted values of a classifier.
- Useful for binary and multiclass classification problems.

In Python

► Code



Accuracy

It is a simple metric for summarizing the information in the confusion matrix. It is the proportion of correct classifications for both classes, out of the total classifications performed.

In Python, we calculate accuracy using the **scikit-learn** `accuracy_score()` function.

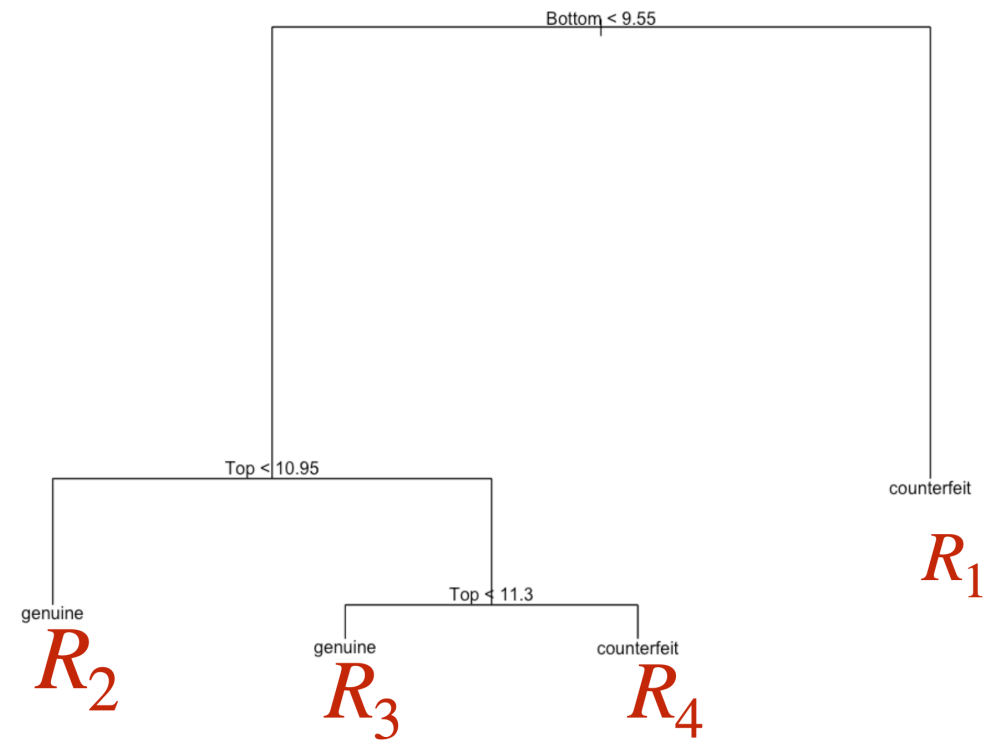
```
1 accuracy = accuracy_score(Y_valid, predicted_class)
2 print( round(accuracy, 2) )
```

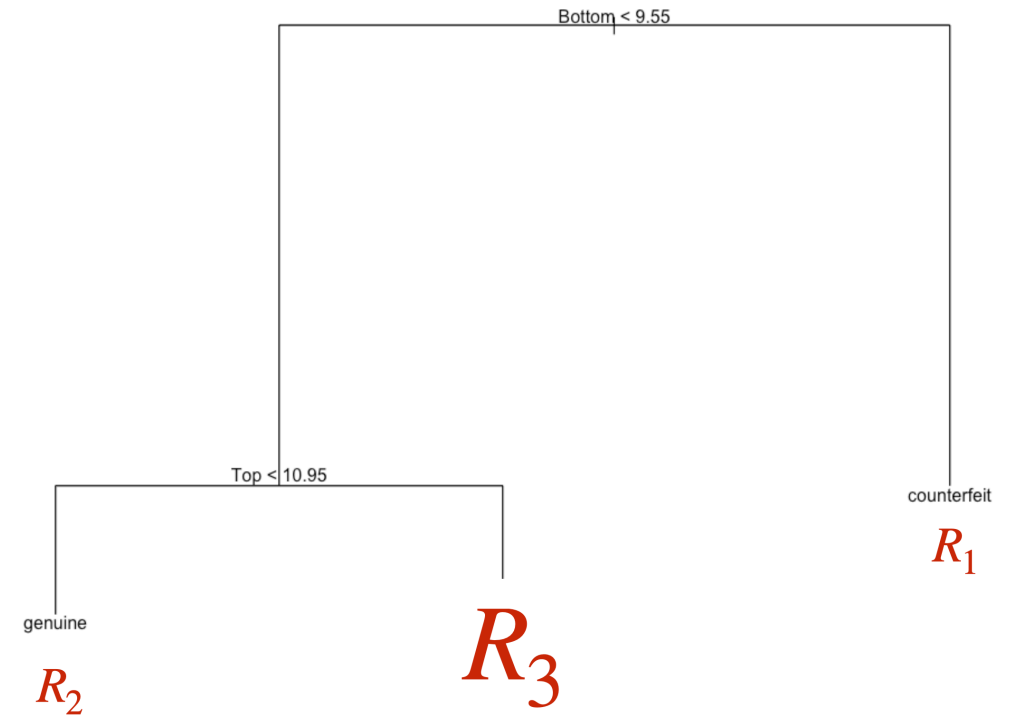
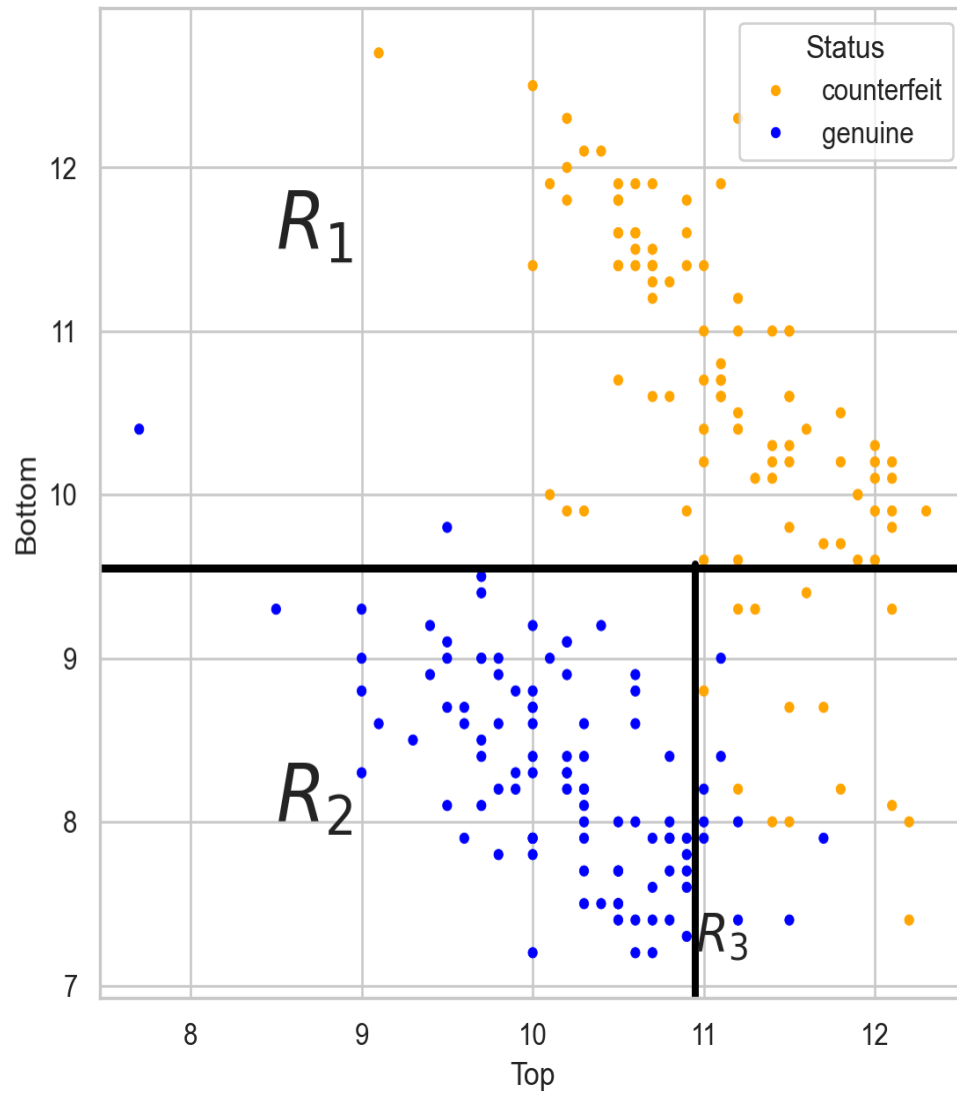
0.97

The higher the accuracy, the better the performance of the classification algorithm.

Pruning the tree

In some cases, we can optimize the performance of the tree by pruning it. That is, we collapse two internal (non-terminal) nodes.





To prune a tree, we use an advanced algorithm to measure the contribution of the tree's branches.

This algorithm minimizes the following function:

Missclassification rate of tree + $\alpha(\# \text{ terminal nodes})$,

where α is a tuning parameter that **places greater weight on the number of tree nodes** (or size).

- Large values of α result in small trees with few nodes.
- Small values of α result in large trees with many nodes.

Apply penalty for large trees

Now, let's illustrate the pruning technique to find a small decision tree that performs well. To do this, we will train several decision trees using different values of α , which weighs the contribution of the tree branches.

In Python, this is achieved using the `cost_complexity_pruning_path()` function with the following syntax.

```
1 path = clf.cost_complexity_pruning_path(X_train, Y_train)
2 ccp_alphas = path.ccp_alphas
```

The `ccp_alphas` object contains the different values of the α parameter used.

Cross-Validation

To compare the performance of the classification tree for the different values of α , we use **5-fold cross validation** as follows.

```
1 # 1. Define the grid of parameters
2 param_grid = {'ccp_alpha': ccp_alphas}
3
4 # 2. Set the classification trees using 5-fold CV.
5 grid_search = GridSearchCV(
6     DecisionTreeClassifier(min_samples_leaf = 5, random_state = 507134),
7     param_grid,
8     cv = 5,
9     scoring = 'accuracy'
10 )
11
12 # 3. Train the classification algorithms
13 grid_search.fit(X_train, Y_train)
```

Now, we extract the results from this process.

```
1 best_alpha = grid_search.best_params_['ccp_alpha']
2 cv_scores = grid_search.cv_results_['mean_test_score']
3
4 print(f"Best Alpha: {best_alpha}")
```

Best Alpha: 0.0

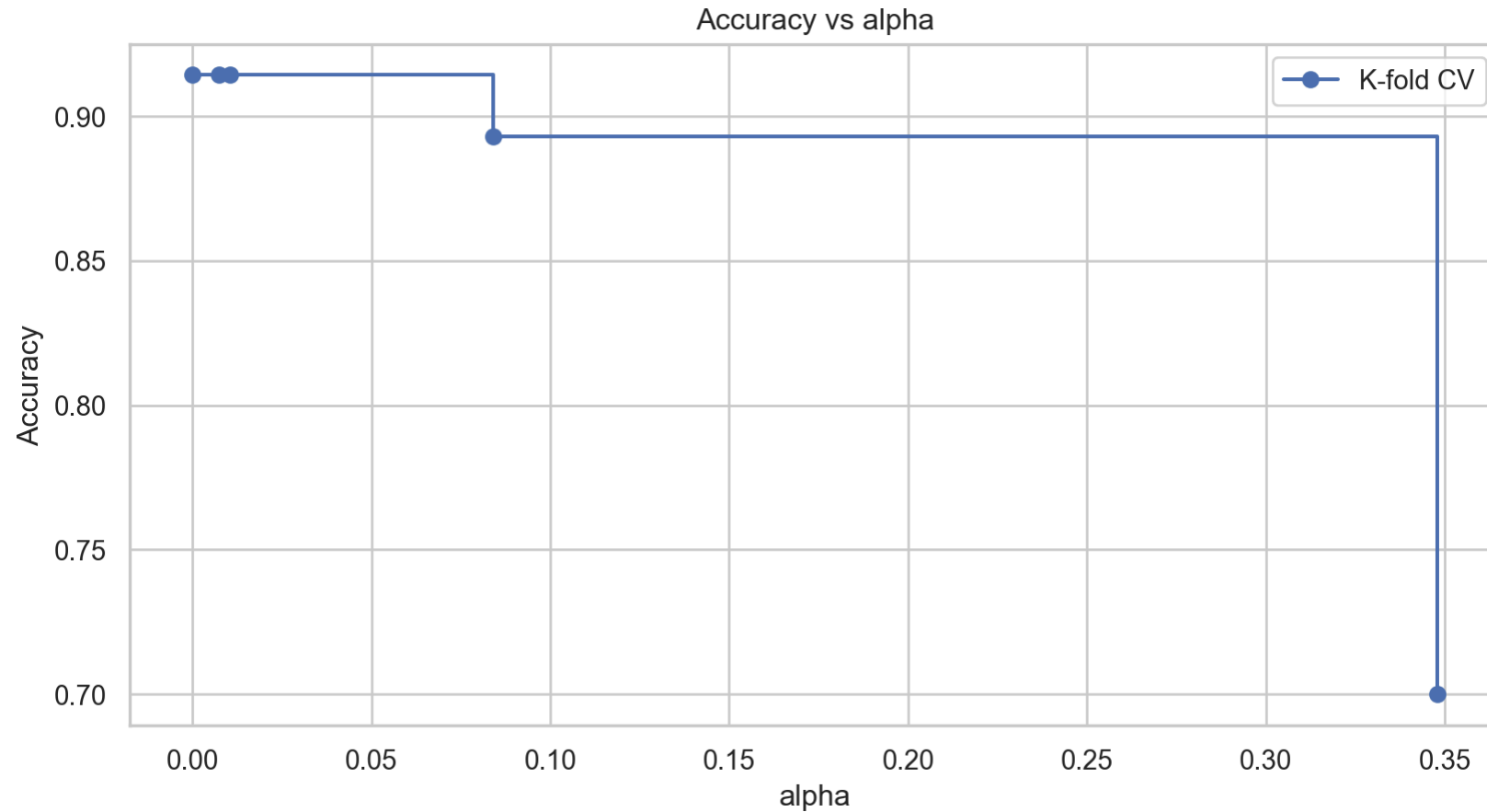
The `best_alpha` is the optimal value of the cost complexity parameter (α). The `cv_scores` shows the estimates of accuracy given by the 5-fold cross-validation.

```
1 cv_scores
```

```
array([0.91428571, 0.91428571, 0.91428571, 0.89285714, 0.7          ])
```

We visualize the results using the following plot.

► Code



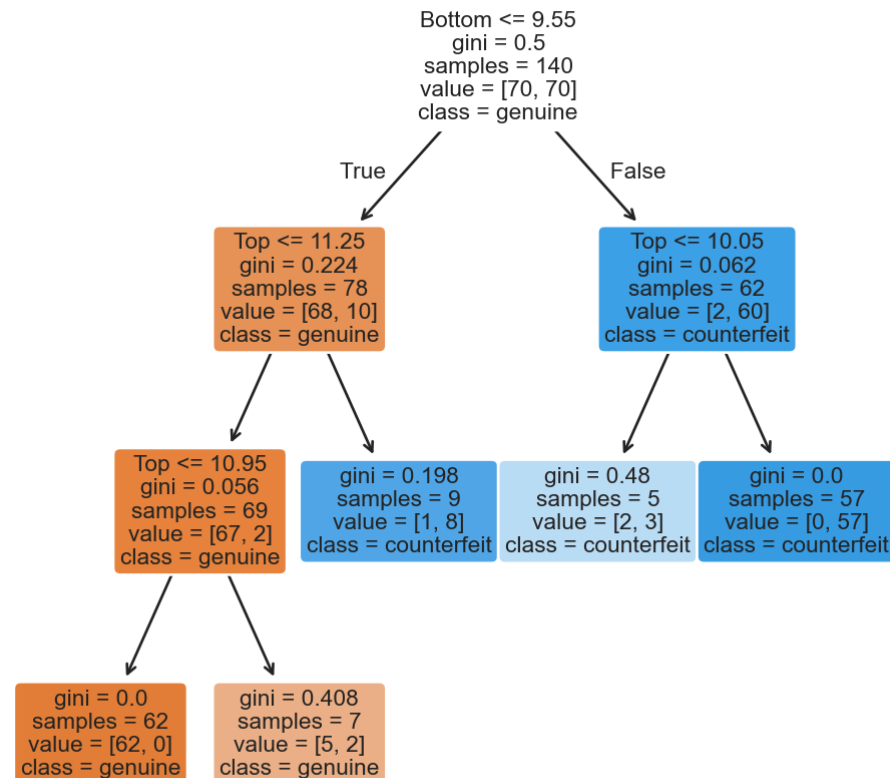
Choosing the best tree

We can see that the best α value for the validation dataset is 0. To train our new reduced tree, we use the `DecisionTreeClassifier()` function again with the `ccp_alpha` parameter set to the best α value. The object containing this new tree is `clf_simple`.

```
1 # We tell Python that we want a classification tree
2 clf_simple = DecisionTreeClassifier(ccp_alpha = best_alpha,
3                                     min_samples_leaf = 5)
4
5 # We train the classification tree using the training data.
6 clf_simple.fit(X_train, Y_train)
```

Once this is done, we can visualize the small tree using the `plot_tree()` function.

► Code



The accuracy of the pruned tree is:

```
1 single_tree_Y_pred = clf_simple.predict(X_valid)
2 accuracy = accuracy_score(Y_valid, single_tree_Y_pred)
3 print( round(accuracy, 2) )
```

0.97

Class-Specific Metrics

Comments on accuracy

- Accuracy is easy to calculate and interpret.
- It works well when the data set has a balanced class distribution (i.e., cases 1 and 0 are approximately equal).
- However, there are situations in which identifying the target class is more important than the reference class.
- For example, it is not ideal for unbalanced data sets. When one class is much more frequent than the other, accuracy can be misleading.

An example

- Let's say we want to create a classifier that tells us whether a mobile phone company's customer will churn next month.
- Customers who churn significantly decrease the company's revenue. That's why it's important to retain these customers.
- To retain that customer, the company will send them a text message with an offer for a low-cost mobile plan.
- Ideally, our classifier correctly identifies customers who will churn, so they get the offer and, hopefully, stay.

- In other words, we want to avoid making wrong decisions about customers who will churn.
- Wrong decisions about loyal customers aren't as relevant.
- Because if we classify a loyal customer as one who will churn, the customer will get a good deal. They'll probably pay less but stay anyway.

Another example

- Another example is developing an algorithm (classifier) that can quickly identify patients who may have a rare disease and need a more extensive and expensive medical evaluation.
- The classifier must make correct decisions about patients with the rare disease, so they can be evaluated and eventually treated.
- A healthy patient who is misclassified with the disease will only incur a few extra dollars to pay for the next test, only to discover that the patient does not have the disease.

Class-specific metrics

To overcome this limitation of accuracy, there are several class-specific metrics. The most popular are:

- **Sensitivity** or *recall*
- **Precision**
- Type I error

These metrics are calculated from the confusion matrix.

Algorithm Prediction

$Y = 1$

$Y = 0$

Validation
Data

$Y = 1$

OO

OR

$Y = 0$

RO

RR

Sensitivity or *recall* = $OO / (OO + OR)$ “How many records of the target class did we predict correctly?”

Algorithm Prediction

$Y = 1$

$Y = 0$

Validation
Data

$Y = 1$

OO

OR

$Y = 0$

RO

RR

Precision = $OO / (OO + RO)$ How many of the records we predicted as target class were classified correctly?

In Python, we compute sensitivity and precision using the following commands:

```
1 recall = recall_score(Y_valid, predicted_class)
2 print( round(recall, 2) )
```

0.97

```
1 precision = precision_score(Y_valid, predicted_class)
2 print( round(precision, 2) )
```

0.97

Algorithm Prediction

$Y = 1$

$Y = 0$

Validation
Data

$Y = 1$

OO

OR

$Y = 0$

RO

RR

Type I error = $RO / (RO + RR)$ “How many of the reference records did we incorrectly predict as targets?”

Unfortunately, there is no simple command to calculate the type-I error in **sklearn**. To overcome this issue, we must calculate it manually.

```
1 # Confusion matrix to compute Type-I error
2 tn, fp, fn, tp = confusion_matrix(Y_valid, predicted_class).ravel()
3
4 # Type-I error rate = False Positive Rate = FP / (FP + TN)
5 type_I_error = fp / (fp + tn)
6 print( round(type_I_error, 2) )
```

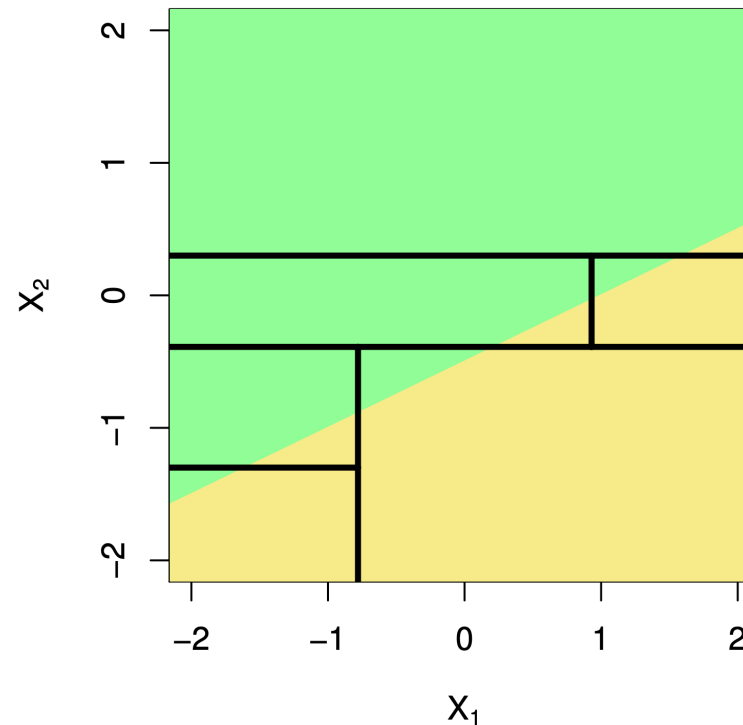
0.03

Discussion

- There is generally a trade-off between sensitivity and Type I error.
- Intuitively, increasing the sensitivity of a classifier is likely to increase Type I error, because more observations are predicted as positive.
- Possible trade-offs between sensitivity and Type I error may be appropriate when there are different penalties or costs associated with each type of error.

Disadvantages of decision trees

- Decision trees have high variance. A small change in the training data can result in a very different tree.
- It has trouble identifying simple data structures.



Return to main page